

```

===== FILE: init_project.m =====
function init_project()
% =====
% 文件名: init_project.m
% 路径: S-Function_14/init_project.m
% 版本号: V1.0
% 最后修改时间: 2025-12-10
% 作者: Auto-generated
% 功能描述:
%   初始化 LPV-MPC 项目运行环境:
%   1) 调用 project_root() 获取根目录
%   2) 将 src 及其子目录递归加入 MATLAB 路径
%   3) 将 simulink 目录加入路径, 便于加载模型
% =====

root = project_root();
addpath(root);
addpath(genpath(fullfile(root, 'src')));
addpath(fullfile(root, 'simulink'));

fprintf('[init_project] Root: %s\n', root);
end

===== FILE: project_root.m =====
function root = project_root()
% =====
% 文件名: project_root.m
% 路径: S-Function_14/project_root.m
% 版本号: V1.0
% 最后修改时间: 2025-12-10
% 作者: Auto-generated
% 功能描述:
%   返回项目根目录的绝对路径, 供所有脚本构建相对路径。
%   该函数应位于项目根目录, 不随其他脚本迁移。
% =====

persistent cached_root
if isempty(cached_root) || ~isfolder(cached_root)
    [cached_root, ~] = fileparts(mfilename('fullpath'));
end
root = cached_root;
end

===== FILE: results_dir.m =====
function dir_out = results_dir(subdir)
% =====
% 文件名: results_dir.m
% 路径: S-Function_14/results_dir.m
% 版本号: V1.0
% 最后修改时间: 2025-12-10
% 作者: Auto-generated
% 功能描述:
%   构建结果输出目录的绝对路径, 并在不存在时自动创建。
%   示例: out_dir = results_dir('gru/performance_offline');
% =====

```

```

if nargin < 1 || isempty(subdir)
    subdir = '';
end

root = project_root();
dir_out = fullfile(root, 'results', subdir);

if ~exist(dir_out, 'dir')
    mkdir(dir_out);
end
end

===== FILE: src/core/agv_model_sfunc.m =====

function agv_model_sfunc(block)
% =====
% Level-2 MATLAB S-Function (Simulink)
% 外壳: AGV 离散植物, 内部调用 state_eq / output_eq
% 接口:
%   - 参数 p: parameters() 结构体 (必须包含 Ts; 可选 nx、x0)
%   - 输入端口(1): u_all = [F_cmd; omega_cmd; theta_ground] (3x1)
%   - 输出端口(1): y (34x1) - V4.3 Mamba 适配
%   - 离散状态: x (nx×1, 默认 8), 使用 DWork 管理
% =====
setup(block);

% ----- S-Function 框架 -----
function setup(block)
% 1) 模块参数
block.NumDialogPrms = 1; % p = parameters()

% 2) 端口
block.NumInputPorts = 1;
block.NumOutputPorts = 1;

% 输入端口: 3×1
block.SetPreCompInPortInfoToDynamic;
block.InputPort(1).Dimensions = 3;
block.InputPort(1).DatatypeID = 0; % double
block.InputPort(1).Complexity = 'Real';
block.InputPort(1).DirectFeedthrough = false;

% 输出端口: 34×1 (V4.3 Mamba 适配)
block.SetPreCompOutPortInfoToDynamic;
block.OutputPort(1).Dimensions = 34;
block.OutputPort(1).DatatypeID = 0; % double
block.OutputPort(1).Complexity = 'Real';

% 3) 采样时间 (来自参数 Ts)
p = block.DialogPrm(1).Data;
if ~isfield(p, 'Ts')
    error('parameters() 必须包含字段 Ts');
end
block.SampleTimes = [p.Ts 0];

```

```

% 4) 不在 setup 里设置 NumDworks / Dwork 属性

% 5) 注册方法
block.SimStateCompliance = 'DefaultSimState';
block.RegBlockMethod('PostPropagationSetup', @DoPostPropSetup);
block.RegBlockMethod('InitializeConditions', @DoInitialize);
block.RegBlockMethod('Outputs', @DoOutputs);
block.RegBlockMethod('Update', @DoUpdate);
end

% ----- 在 PostPropagationSetup 里设置 DWork 的数量和属性 -----
function DoPostPropSetup(block)
    block.NumDworks = 1; % 关键: 必须在这里设置

    % 状态维度 (可用 p.nx 指定, 默认 8)
    nx = 8;
    if block.NumDialogPrms>=1 && isstruct(block.DialogPrm(1).Data)
        p = block.DialogPrm(1).Data;
        if isfield(p,'nx') && ~isempty(p.nx)
            nx = p.nx;
        end
    end

    block.Dwork(1).Name = 'xdisc';
    block.Dwork(1).Dimensions = nx;
    block.Dwork(1).DatatypeID = 0; % double
    block.Dwork(1).Complexity = 'Real';
    block.Dwork(1).UsedAsDiscState = true; % 离散状态
end

% ----- 初始化离散状态 -----
function DoInitialize(block)
    p = block.DialogPrm(1).Data;
    nx = block.Dwork(1).Dimensions;

    % 优先使用 p.x0; 否则由 parameters.m 的 initial_* 字段组装
    if isfield(p,'x0') && ~isempty(p.x0)
        x0 = p.x0(:);
    elseif all(isfield(p, {'initial_x','initial_y','initial_heading', ...
        'initial_velocity','initial_angular_velocity', ...
        'initial_front_steering','initial_rear_steering', ...
        'initial_sideslip'}))
        x0 = [p.initial_x; p.initial_y; p.initial_heading; ...
            p.initial_velocity; p.initial_angular_velocity; ...
            p.initial_front_steering; p.initial_rear_steering; ...
            p.initial_sideslip];
    else
        x0 = zeros(nx,1);
    end

    if numel(x0)~=nx
        error('x0 维度(%d)与 nx(%d)不一致', numel(x0), nx);
    end
    block.Dwork(1).Data = x0;
end

```

```

end

% ----- 输出方程 -----
function DoOutputs(block)
    p      = block.DialogPrm(1).Data;
    u_all  = block.InputPort(1).Data;      % [F_cmd; omega_cmd; theta_ground]
    x      = block.Dwork(1).Data;

    u      = u_all(1:2);                    % 控制量
    theta_ground = u_all(3);                % 坡度角

    % y = output_eq(x, u, theta_ground, p); % 修正: 按签名传入 4 个参数
    y = output_eq_ref(x, u, theta_ground, p); % 利用 v_ref 和 omega_ref 生成参考半径

    if ~isvector(y) || numel(y)~=block.OutputPort(1).Dimensions
        % error('output_eq 返回长度与输出端口不一致, 期望 %d, 实际 %d', ... % 利用 v_ref 和 omega_ref 生成参考半径
        % block.OutputPort(1).Dimensions, numel(y));
        error('output_eq_ref 返回长度与输出端口不一致, 期望 %d, 实际 %d', ...
            block.OutputPort(1).Dimensions, numel(y));
    end
    block.OutputPort(1).Data = y(:);
end

% ----- 状态更新 (离散化步进) -----
function DoUpdate(block)
    p      = block.DialogPrm(1).Data;
    u_all  = block.InputPort(1).Data;      % [F_cmd; omega_cmd; theta_ground]
    x      = block.Dwork(1).Data;

    u      = u_all(1:2);
    theta_ground = u_all(3);

    % 修正: state_eq 的签名为 (x, u, theta_ground, params)
    % x_next = state_eq(x, u, theta_ground, p); % 利用 v_ref 和 omega_ref 生成参考半径
    x_next = state_eq_ref(x, u, theta_ground, p);

    if numel(x_next)~=numel(x)
        % error('state_eq 返回维度与 DWork 不一致'); % 利用 v_ref 和 omega_ref 生成参考半径
        error('state_eq_ref 返回维度与 DWork 不一致');
    end
    block.Dwork(1).Data = x_next(:);
end

===== FILE: src/core/agv_model_sfunc_train_data.m =====
function agv_model_sfunc_train_data(block)
%AGV_MODEL_SFUNC_TRAIN_DATA 训练数据生成专用 Level-2 MATLAB S-Function.
%
% 功能说明:
% 本模块由 GRU_DataGen.slx 调用, 用于生成 GRU/TCN 共享训练数据。
% 它与闭环主模型 agv_model_sfunc.m 分离, 原因是训练数据生成需要
% 额外注入滑移和负载/堵转扰动, 不能污染正式控制模型。
%
% 输入端口:

```

```

% u_all = [F_cmd; omega_cmd; theta_ground; slip_gamma; stall_load]
%
% F_cmd      : 纵向驱动力命令 [N]
% omega_cmd  : 横摆角速度命令 [rad/s]
% theta_ground: 地面坡度角 [rad]
% slip_gamma : 滑移注入系数, 1.0 表示正常附着
% stall_load : 外加纵向负载, 用于 load/stall 扰动注入 [N]
%
% 内部动力学/输出方程:
% state_eq_ref_train_data(...)
% output_eq_ref_train_data(...)
%
% 输出约定:
% y 为 34 x 1, 保持当前 GRU/Mamba/TCN 数据生成链路使用的训练信号
% 契约。第 32-34 维为滑移率和横向加速度等扩展诊断量。

setup(block);

function setup(block)
    block.NumDialogPrms = 1; % params 参数结构体

    block.NumInputPorts = 1;
    block.NumOutputPorts = 1;

    block.SetPreCompInPortInfoToDynamic;
    block.InputPort(1).Dimensions = 5;
    block.InputPort(1).DatatypeID = 0; % double 类型
    block.InputPort(1).Complexity = 'Real';
    block.InputPort(1).DirectFeedthrough = false;

    block.SetPreCompOutPortInfoToDynamic;
    block.OutputPort(1).Dimensions = 34;
    block.OutputPort(1).DatatypeID = 0; % double 类型
    block.OutputPort(1).Complexity = 'Real';

    p = block.DialogPrm(1).Data;
    if ~isstruct(p) || ~isfield(p, 'Ts')
        error('agv_model_sfunc_train_data:InvalidParams', ...
            'Dialog parameter must be a params struct with field Ts.');
```

end

```

    block.SampleTimes = [p.Ts 0];

    block.SimStateCompliance = 'DefaultSimState';
    block.RegBlockMethod('PostPropagationSetup', @DoPostPropSetup);
    block.RegBlockMethod('InitializeConditions', @DoInitialize);
    block.RegBlockMethod('Outputs', @DoOutputs);
    block.RegBlockMethod('Update', @DoUpdate);
end

function DoPostPropSetup(block)
    nx = 8;
    if block.NumDialogPrms >= 1 && isstruct(block.DialogPrm(1).Data)
        p = block.DialogPrm(1).Data;
        if isfield(p, 'nx') && ~isempty(p.nx)
            nx = p.nx;
        end
    end
end

```

```

        end
    end

    block.NumDworks = 1;
    block.Dwork(1).Name = 'xdisc';
    block.Dwork(1).Dimensions = nx;
    block.Dwork(1).DatatypeID = 0;
    block.Dwork(1).Complexity = 'Real';
    block.Dwork(1).UsedAsDiscState = true;
end

function DoInitialize(block)
    p = block.DialogPrm(1).Data;
    nx = block.Dwork(1).Dimensions;

    if isfield(p, 'x0') && ~isempty(p.x0)
        x0 = p.x0(:);
    elseif all(isfield(p, {'initial_x', 'initial_y', 'initial_heading', ...
        'initial_velocity', 'initial_angular_velocity', ...
        'initial_front_steering', 'initial_rear_steering', ...
        'initial_sideslip'}))
        x0 = [p.initial_x; p.initial_y; p.initial_heading; ...
            p.initial_velocity; p.initial_angular_velocity; ...
            p.initial_front_steering; p.initial_rear_steering; ...
            p.initial_sideslip];
    else
        x0 = zeros(nx, 1);
    end

    if numel(x0) ~= nx
        error('agv_model_sfunc_train_data:InvalidX0', ...
            'x0 length (%d) does not match nx (%d).', numel(x0), nx);
    end
    block.Dwork(1).Data = x0;
end

function DoOutputs(block)
    p = block.DialogPrm(1).Data;
    u_all = block.InputPort(1).Data;
    x = block.Dwork(1).Data;

    u = u_all(1:2);
    theta_ground = u_all(3);
    slip_gamma = u_all(4);
    stall_load = u_all(5);

    y = output_eq_ref_train_data(x, u, theta_ground, p, slip_gamma, stall_load);

    if ~isvector(y) || numel(y) ~= block.OutputPort(1).Dimensions
        error('agv_model_sfunc_train_data:InvalidOutputSize', ...
            'output_eq_ref_train_data returned %d elements, expected %d.', ...
            numel(y), block.OutputPort(1).Dimensions);
    end
    block.OutputPort(1).Data = y(:);
end

```

```

function DoUpdate(block)
    p = block.DialogPrm(1).Data;
    u_all = block.InputPort(1).Data;
    x = block.Dwork(1).Data;

    u = u_all(1:2);
    theta_ground = u_all(3);
    slip_gamma = u_all(4);
    stall_load = u_all(5);

    x_next = state_eq_ref_train_data(x, u, theta_ground, p, slip_gamma, stall_load);

    if numel(x_next) ~= numel(x)
        error('agv_model_sfunc_train_data:InvalidStateSize', ...
            'state_eq_ref_train_data returned %d elements, expected %d.', ...
            numel(x_next), numel(x));
    end
    block.Dwork(1).Data = x_next(:);
end

end

===== FILE: src/core/state_eq.m =====
function x_next = state_eq(x, u, theta_ground, params)
% =====
% 文件名: state_eq.m
% 功能描述: 对角式双舵轮 AGV 状态转移方程 - 实现车辆动力学模型的时间步进
% 输入参数:
%   x: 当前时刻状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
%       其中: X-全局 X 坐标[m], Y-全局 Y 坐标[m], psi-航向角[rad]
%             v-纵向速度[m/s], omega-横摆角速度[rad/s]
%             delta_lf-左前轮转向角[rad], delta_rr-右后轮转向角[rad]
%             beta-质心侧偏角[rad]
%   u: 控制输入向量 [F_cmd omega_cmd] (2×1)
%       F_cmd-总驱动力指令[N], omega_cmd-期望横摆角速度[rad/s]
%   theta_ground: 地面坡度角[rad] (标量)
%   params: 包含所有车辆参数的结构体
% 输出参数:
%   x_next: 下一时刻状态向量 (8×1)
% =====

%% 提取状态变量 - 从状态向量中分解各个状态分量
X = x(1); Y = x(2); psi = x(3); v = x(4);
omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);

%% 提取控制输入 - 从输入向量分解控制指令
F_cmd = u(1); % 总驱动力指令[N]
omega_cmd = u(2); % 期望横摆角速度指令[rad/s]

%% 直接从参数结构体获取车辆参数 - 提高代码可读性和维护性
Ts = params.Ts; % 采样时间[s]
m = params.mass; % 车辆总质量[kg]
I_z = params.Iz; % 绕 Z 轴的转动惯量[kg·m²]
L = params.L; % 轴距[m]

```

```

W = params.W; % 轮距[m]
h_cg = params.h_cg; % 质心高度[m]
r = params.wheel_radius; % 车轮半径[m]
n = params.gear_ratio; % 传动比
eta = params.gear_efficiency; % 传动效率
k_t = params.motor_torque_constant; % 电机转矩常数[N·m/A]
mu = params.friction_coefficient; % 路面摩擦系数
c_r = params.rolling_resistance; % 滚动阻力系数
rho = params.air_density; % 空气密度[kg/m³]
CdA = params.drag_coefficient_area; % 风阻系数×迎风面积[m²]
C_af = params.front_cornering_stiffness; % 前轮侧偏刚度[N/rad]
C_ar = params.rear_cornering_stiffness; % 后轮侧偏刚度[N/rad]
delta_max = params.max_steering_angle; % 最大转向角限制[rad]
delta_dot_max = params.max_steering_rate; % 最大转向速率[rad/s]
tau_delta = params.steering_time_constant; % 转向系统时间常数[s]
current_limit = params.current_limit; % 电机电流限制[A]
accel_limit = params.max_acceleration; % 最大加速度限制[m/s²]
g = params.gravity; % 重力加速度[m/s²]
min_omega_threshold = params.min_angular_velocity_threshold; % 最小角速度阈值[rad/s]
low_speed_thresh = params.low_speed_threshold; % 低速阈值[m/s]

%% === 转向几何计算: 按 ICR 约束 (tanδ 关系) 生成目标角 ===
if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh
    % 低速/小指令: 维持直行
    delta_lf_target = 0; delta_rr_target = 0;
else
    % 角速度误差抑制 (比例缩放, 保持两轮几何比不变)
    e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
    if abs(omega) > abs(omega_cmd) * 1.2 && sign(omega) == sign(omega_cmd)
        delta_scale = max(omega_cmd / max(abs(omega), 1e-6), 0.1);
    else
        delta_scale = 1.0;
    end

    % ICR 坐标 (车辆质心坐标系): 改用测量 ω (或与命令加权的 ω) 计算
    % 目的: 避免在 v 下降或跟踪误差较大时, 把目标舵角放大
    w_icr = omega; % 如需更强前馈, 可用 w_icr = 0.7*omega + 0.3*omega_cmd;
    R_cmd = (v / max(abs(w_icr), 1e-6)) * sign(w_icr); % 带符号半径[m]
    x_c = 0.0; % 对称转弯, ICR 在 y 轴上
    y_c = R_cmd; % ICR y 坐标 (可正可负)

    % 几何约束:
    % tan(delta_lf) = (L/2 - x_c) / (y_c - W/2)
    % tan(delta_rr) = -(x_c + L/2) / (y_c + W/2)
    delta_lf_geom = atan2((L/2 - x_c), (y_c - W/2));
    delta_rr_geom = -atan2((x_c + L/2), (y_c + W/2));

    % 误差缩放 (不破坏两轮几何比例)
    delta_lf_target = delta_lf_geom * delta_scale;
    delta_rr_target = delta_rr_geom * delta_scale;

    % 取消额外的“安全夹紧”: 由执行器限幅 delta_max 与速率限幅保障
    % 这样可避免稳态曲率下目标转角长期贴住 15° 而过大
end
end

```



```

%% === 舵机动力学模拟：一阶惯性 + 速率限制 + 角度限制 ===
% 转向系统动力学：一阶惯性环节模拟转向执行器响应
delta_lf_dot = (delta_lf_target - delta_lf)/max(tau_delta,Ts);
delta_rr_dot = (delta_rr_target - delta_rr)/max(tau_delta,Ts);

% 转向速率限制 - 模拟转向执行器的物理限制
delta_lf_dot = sat(delta_lf_dot, -delta_dot_max, delta_dot_max);
delta_rr_dot = sat(delta_rr_dot, -delta_dot_max, delta_dot_max);

% 转向角更新 + 角度限制
delta_lf_new = sat(delta_lf + Ts*delta_lf_dot, -delta_max, delta_max);
delta_rr_new = sat(delta_rr + Ts*delta_rr_dot, -delta_max, delta_max);

%% === 载荷转移计算 & 滚动阻力计算 ===
[N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground, params);

%% === 电机/传动系统限制推算到车轮牵引力上限 ===
% 计算单个驱动轮的最大牵引力（基于电机电流限制）
F_wheel_max = current_limit * k_t * (eta * n) / max(r,1e-6); % 单驱动轮最大牵引力[N]

% 计算总驱动轮的最大牵引力（对角式：左前+右后两个驱动轮）
num_drive_wheels = 2; % 驱动轮数量：lf & rr
F_cmd_max_total = num_drive_wheels * F_wheel_max;

% 应用牵引力限制到指令值
F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);

%% === 轮胎侧偏角计算（基于简化自行车模型） ===
% 车辆几何参数：假设质心位于轴距中心
Lf = L/2; Lr = L/2; % 前后轴到质心的距离[m]

% 侧偏刚度：使用标称值（取消转弯增强的临时放大）
% C_af, C_ar 已由 parameters.m 提供

% 纵向速度分量（防止除零）
v_x = max(v * cos(beta), 1e-3);

% 前轮侧偏角（标准符号）：alpha_f = (beta + Lf*omega/v) - delta_lf
alpha_f = (beta + Lf*omega/max(v,low_speed_thresh)) - delta_lf_new;

% 后轮侧偏角（标准符号）：alpha_r = (beta - Lr*omega/v) - delta_rr
alpha_r = (beta - Lr*omega/max(v,low_speed_thresh)) - delta_rr_new; % 对角式双舵轮，后轮也有转向

% 线性侧偏力计算 + 软饱和（限幅到摩擦圆边界）
Fy_f_lin = -C_af * alpha_f; % 前轮线性侧偏力[N]
Fy_r_lin = -C_ar * alpha_r; % 后轮线性侧偏力[N]

% 基于垂向载荷计算最大侧向力（摩擦圆限制）
Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]

% 应用侧向力饱和限制
Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);

```

```

%% === 牵引力分配策略：按法向载荷比例 + 偏航控制 ===
% 驱动轮总垂向载荷（仅考虑两个驱动轮：左前+右后）
W_drive = N_lf + N_rr; % 驱动轮总载荷[N]

% 计算各驱动轮在总驱动载荷中的权重比例
w_lf = N_lf / max(W_drive, 1e-6); % 左前轮载荷占比
w_rr = N_rr / max(W_drive, 1e-6); % 右后轮载荷占比

% 直线/低曲率时等分扭矩，抑制对角驱动导致的偏航力矩
if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
    w_lf = 0.5;
    w_rr = 0.5;
end

% 偏航控制：根据角速度误差产生驱动力差以实现横摆控制
e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]

% 横摆角速度保护：防止角速度爆炸式增长
omega_max = 2.0; % 最大角速度限制 [rad/s]（远大于目标 0.1 rad/s）

if v < low_speed_thresh || abs(omega_cmd) < min_omega_threshold
    % 低速/小指令：关断偏航力差，避免原地自旋
    Delta_Fx = 0;
else
    if abs(omega) > omega_max
        % 情况：角速度超限 - 施加强制制动力矩进行保护
        omega_sign = sign(omega);
        Mz_brake = -omega_sign * I_z * 50.0; % 强制制动力矩[Nm]
        Delta_Fx = 2 * Mz_brake / max(W, 1e-3); % 所需的驱动力差[N]
    else
        % 恢复：正常 P 控制产生偏航力差
        K_omega_p = 50.0;
        omega_dot_desired = K_omega_p * e_omega; % 期望角加速度[rad/s²]
        omega_dot_max = 10.0; % 最大角加速度 [rad/s²]
        omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
        Mz_needed = I_z * omega_dot_desired; % 所需偏航力矩 [Nm]
        Delta_Fx = 2 * Mz_needed / max(W, 1e-3); % 所需驱动力差 [N]
        Delta_Fx_max = max(abs(F_cmd_eff), 10) * 0.5; % 驱动力差限幅
        Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);
    end
end

% 驱动力分配：基础分配（按载荷比例）+ 偏航控制（力差调整）
Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2; % 左前轮指令驱动力[N]
Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2; % 右后轮指令驱动力[N]

%% === 摩擦椭圆限幅：确保合力不超过轮胎-路面摩擦极限 ===
% 获取是否使用光滑限幅的标志
use_smooth = params.use_smooth_saturation;

if use_smooth
    % 光滑版本限幅：使用 softplus 函数替代 max(·,0)，提高数值稳定性
    smooth_gain = params.smooth_gain;

    % 计算各轮胎在摩擦椭圆中的可用纵向力余量

```

```

term_lf = softplus(1 - (Fy_f/max(mu*N_lf,1e-6))^2, smooth_gain);
term_rr = softplus(1 - (Fy_r/max(mu*N_rr,1e-6))^2, smooth_gain);

Fx_lf_allow = mu * N_lf * sqrt(term_lf); % 左前轮允许的最大纵向力[N]
Fx_rr_allow = mu * N_rr * sqrt(term_rr); % 右后轮允许的最大纵向力[N]

% 应用光滑限幅到指令驱动力
F_drive_lf = sat_smooth(Fx_lf_cmd, -Fx_lf_allow, Fx_lf_allow, smooth_gain);
F_drive_rr = sat_smooth(Fx_rr_cmd, -Fx_rr_allow, Fx_rr_allow, smooth_gain);
else
% 硬限幅版本: 传统的摩擦椭圆限制
Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf,1e-6))^2, 0));
Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr,1e-6))^2, 0));

F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);
end

% 计算总有效驱动力
F_drive_total = F_drive_lf + F_drive_rr;

% 计算驱动不对称产生的偏航力矩 (左右驱动力差异引起)
Mz_drive = (W/2) * (F_drive_lf - F_drive_rr);

% 坡度阻力计算
F_slope = m * g * sin(theta_ground);

% 有效质量计算 (包含旋转部件的等效质量)
wheel_inertia = params.wheel_inertia; % 车轮转动惯量[kg·m²]
motor_inertia = params.motor_inertia; % 电机转动惯量[kg·m²]
m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2); % 总有效质量[kg]

%% === RK4 数值积分 (转向角线性插值 + 动态空气阻力计算) ===
% 提取核心状态变量用于 RK4 积分
s_k = [X; Y; psi; v; omega; beta];

% 转向角中点值 (用于 RK4 中间步骤的线性插值)
delta_lf_mid = (delta_lf + delta_lf_new)/2;
delta_rr_mid = (delta_rr + delta_rr_new)/2;

% 获取光滑限幅参数
use_smooth = params.use_smooth_saturation;
smooth_gain = params.smooth_gain;

% 定义连续动力学核心函数 (闭包形式, 传入当前计算的所有参数)
CONT_CORE = @(s, dl, dr) continuous_dynamics_core_v2(s, dl, dr, ...
    F_drive_lf, F_drive_rr, F_rolling_total, F_slope, m_eff_total, I_z, Lf, Lr, ...
    low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
    rho, CdA); % 传入空气阻力参数用于动态计算

% RK4 积分四步计算 (使用转向角插值)
k1 = CONT_CORE(s_k, delta_lf, delta_rr); % 第一步: 使用 k 时刻转向角
k2 = CONT_CORE(s_k + 0.5*Ts*k1, delta_lf_mid, delta_rr_mid); % 第二步: 使用中点转向角
k3 = CONT_CORE(s_k + 0.5*Ts*k2, delta_lf_mid, delta_rr_mid); % 第三步: 使用中点转向角
k4 = CONT_CORE(s_k + Ts*k3, delta_lf_new, delta_rr_new); % 第四步: 使用 k+1 时刻转向角

```

```

% RK4 最终积分: 加权平均
s_new = s_k + (Ts/6)*(k1 + 2*k2 + 2*k3 + k4);

%% 提取新状态并应用保护措施
X_new = s_new(1); Y_new = s_new(2); psi_new = s_new(3);
v_new = s_new(4); omega_new = s_new(5); beta_new = s_new(6);

% 航向角归一化到[-pi, pi]范围
psi_new = normalizeAngle(psi_new);

% 低速保护: 防止速度变为负值(车辆不能倒车)
if v_new < 0
    v_new = 0;
end

% 侧偏角限幅保护(防止数值爆炸)
beta_new = sat(beta_new, -deg2rad(15), deg2rad(15));

%% 组装下一时刻完整状态向量
x_next = [X_new; Y_new; psi_new; v_new; omega_new; delta_lf_new; delta_rr_new; beta_new];

end

% ===== 连续动力学核心函数 V2(动态空气阻力计算) =====
function ds = continuous_dynamics_core_v2(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_rolling_total, F_slope, ...
    m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
    rho, CdA)
% 连续动力学核心函数 V2 - 用于 RK4 积分的每个子步, 动态计算空气阻力
% 输入参数:
%   s: 状态向量 [X Y psi v omega beta]
%   delta_lf, delta_rr: 当前转向角[rad]
%   Fx_lf, Fx_rr: 预计算的驱动力(已应用摩擦椭圆限幅)[N]
%   F_rolling_total: 总滚动阻力[N]
%   F_slope: 坡度阻力[N]
%   m_eff_total: 总有效质量[kg]
%   I_z: 转动惯量[kg·m²]
%   Lf, Lr: 前后轴到质心距离[m]
%   low_speed_thresh: 低速阈值[m/s]
%   C_af, C_ar: 前后轮侧偏刚度[N/rad]
%   mu: 摩擦系数
%   N_lf, N_rr: 左右驱动轮垂向载荷[N]
%   W: 轮距[m]
%   use_smooth: 是否使用光滑限幅
%   smooth_gain: 光滑限幅增益
%   rho: 空气密度[kg/m³]
%   CdA: 风阻系数×面积[m²]
% 输出参数:
%   ds: 状态导数向量 [X_dot Y_dot psi_dot v_dot omega_dot beta_dot]

% 设置默认参数值
if nargin < 22
    use_smooth = false;
    smooth_gain = 30.0;
end

```

```

% 分解状态向量
X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);

% 动态计算空气阻力（基于当前瞬时速度）
F_aero = 0.5 * rho * CdA * v^2 * sign(v); % 空气阻力[N], 与速度平方成正比
F_drag = F_rolling_total + F_aero; % 总阻力 = 滚动阻力 + 空气阻力

% 重新计算轮胎侧偏角（基于当前状态和转向角）
alpha_f = (beta + Lf*omega/max(abs(v),low_speed_thresh)) - delta_lf;
alpha_r = (beta - Lr*omega/max(abs(v),low_speed_thresh)) - delta_rr;

% 线性侧偏力计算
Fy_f_lin = -C_af * alpha_f; % 前轮侧偏力[N]
Fy_r_lin = -C_ar * alpha_r; % 后轮侧偏力[N]

% 侧向力饱和限制（基于摩擦圆原理）
Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]

% 应用侧向力限幅（可选择光滑或硬限幅）
if use_smooth
    Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
    Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
else
    Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
    Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
end

% 使用预计算的驱动力（避免重复限幅）
F_drive_actual = Fx_lf + Fx_rr; % 实际总驱动力[N]

% 驱动不对称产生的偏航力矩
Mz_drive = (W/2) * (Fx_lf - Fx_rr); % [Nm]

% 纵向动力学方程
a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total,1e-6); % 纵向加速度[m/s²]

% 低速保护：防止倒车情况下的负加速度
if a_x < 0 && v < 0
    v_dot = 0; % 速度为负时不允许进一步减速
else
    v_dot = a_x; % 正常加速度
end

% 横摆动力学计算
Mz_tire = Fy_f*Lf - Fy_r*Lr; % 轮胎侧偏力产生的偏航力矩[Nm]
Mz_total = Mz_tire + Mz_drive; % 总偏航力矩 = 轮胎力矩 + 驱动力矩

% 恢复：横摆阻尼回到标称
C_damping = 1000.0; % 横摆阻尼系数 [Nm/(rad/s)]
Mz_damping = -C_damping * omega; % 阻尼力矩（与角速度成正比）[Nm]
Mz_total = Mz_total + Mz_damping; % 包含阻尼的总偏航力矩[Nm]

% 角加速度计算

```

```

omega_dot = Mz_total / max(I_z,1e-6); % [rad/s²]

% 角加速度限幅保护
omega_dot_limit = 5.0; % 最大角加速度限制 [rad/s²]
omega_dot = sat_sym(omega_dot, omega_dot_limit);

% 侧滑角动力学（添加强阻尼确保稳定性）
if abs(v) < low_speed_thresh
    % 低速情况：使用强阻尼强制侧滑角收敛到零
    beta_dot = -5.0*beta;
else
    % 正常速度：基于力学原理的侧滑角动力学 + 强阻尼项
    beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v),low_speed_thresh/10)) - omega - 5.0*beta;
end

% 侧滑角变化率限幅
beta_dot_limit = deg2rad(10); % 限制侧滑角变化率[rad/s]
beta_dot = sat_sym(beta_dot, beta_dot_limit);

% 运动学方程
psi_dot = omega; % 航向角变化率[rad/s]
X_dot = v * cos(psi + beta); % X 方向速度[m/s]
Y_dot = v * sin(psi + beta); % Y 方向速度[m/s]

% 组装状态导数向量
ds = [X_dot; Y_dot; psi_dot; v_dot; omega_dot; beta_dot];

% 局部饱和函数定义
function y = sat(x, xmin, xmax)
    y = min(max(x, xmin), xmax);
end

% ===== 连续动力学核心函数（原版，保留兼容性） =====
function ds = continuous_dynamics_core(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_drag, F_slope, ...
    m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain)
% 连续动力学核心函数（原版）- 用于 RK4 积分，使用固定的阻力值
% 输入参数说明同 V2 版本，但 F_drag 为固定值而非动态计算

% 设置默认参数
if nargin < 20
    use_smooth = false;
    smooth_gain = 30.0;
end

% 状态分解
X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);

% 侧偏角计算
alpha_f = delta_lf - (beta + Lf*omega/max(abs(v),1e-3));
alpha_r = delta_rr - (beta - Lr*omega/max(abs(v),1e-3));

% 侧向力计算与限幅
Fy_f_lin = -C_af * alpha_f;
Fy_r_lin = -C_ar * alpha_r;

```

```

Fy_f_max = mu * N_lf;
Fy_r_max = mu * N_rr;

if use_smooth
    Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
    Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
else
    Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
    Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
end

% 驱动力和偏航力矩
F_drive_actual = Fx_lf + Fx_rr;
Mz_drive = (W/2) * (Fx_lf - Fx_rr);

% 纵向动力学
a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total, 1e-6);

% 低速保护
if a_x < 0 && v < 0
    v_dot = 0;
else
    v_dot = a_x;
end

% 横摆动力学
Mz_tire = Fy_f*Lf - Fy_r*Lr;
Mz_total = Mz_tire + Mz_drive;

% 横摆阻尼
C_damping = 1000.0;
Mz_damping = -C_damping * omega;
Mz_total = Mz_total + Mz_damping;

omega_dot = Mz_total / max(I_z, 1e-6);

% 角加速度限幅
omega_dot_limit = 5.0;
omega_dot = sat_sym(omega_dot, omega_dot_limit);

% 侧滑角动力学
if abs(v) < low_speed_thresh
    beta_dot = -5.0*beta;
else
    beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v), low_speed_thresh/10)) - omega - 5.0*beta;
end

% 侧滑角变化率限幅
beta_dot_limit = deg2rad(10);
beta_dot = sat_sym(beta_dot, beta_dot_limit);

% 运动学
psi_dot = omega;
X_dot = v * cos(psi + beta);

```

```

Y_dot = v * sin(psi + beta);

% 输出状态导数
ds = [X_dot; Y_dot; psi_dot; v_dot; omega_dot; beta_dot];

% 局部饱和函数
function y = sat(x, xmin, xmax)
    y = min(max(x, xmin), xmax);
end

end

%% ===== 工具函数集 =====
function y = sat(x, xmin, xmax)
% 标准饱和函数 - 将输入 x 限制在 [xmin, xmax] 范围内
% 输入: x-输入值, xmin-最小值, xmax-最大值
% 输出: y-饱和后的值
y = min(max(x, xmin), xmax);
end

function y = sat_sym(x, xmax)
% 对称饱和函数 - 将输入 x 限制在 [-xmax, xmax] 范围内
% 输入: x-输入值, xmax-对称的最大绝对值
% 输出: y-饱和后的值
y = min(max(x, -xmax), xmax);
end

function y = sat_smooth(x, xmin, xmax, k)
% 光滑饱和函数 - 使用双曲正切实现光滑限幅
% 输入: x-输入值, xmin-最小值, xmax-最大值, k-光滑增益 (越大越接近硬限幅)
% 输出: y-光滑饱和后的值
if nargin < 4
    k = 30.0; % 默认光滑增益
end
if xmax <= xmin
    y = (xmin + xmax) / 2; % 异常情况处理
    return;
end
% 归一化处理 + 双曲正切光滑限幅
x_norm = (x - xmin) / (xmax - xmin); % 归一化到[0,1]
y_norm = (tanh(k * (x_norm - 0.5)) + 1) / 2; % 光滑 S 曲线变换
y = xmin + (xmax - xmin) * y_norm; % 反归一化
end

function y = softplus(x, k)
% 光滑的 max(x,0) 近似函数 - 提高数值稳定性
% 输入: x-输入值, k-光滑增益 (越大越接近 max 函数)
% 输出: y-光滑处理后的值
if nargin < 2
    k = 30.0; % 默认光滑增益
end
y = (1/k) * log(1 + exp(k * x)); % softplus 函数公式
end

function a = normalizeAngle(a)
% 角度归一化函数 - 将角度包装到 [-pi, pi] 范围内

```



```

% 输入: a-输入角度[rad]
% 输出: a-归一化后的角度[rad]
a = atan2(sin(a), cos(a)); % 使用 atan2 实现稳健的角度归一化
end

function [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_g, params)
% 载荷转移计算函数 - 基于纵向/横向加速度的简化载荷转移模型 + 坡度影响
% 输入参数:
%   F_cmd: 驱动力指令[N], omega: 横摆角速度[rad/s], v: 速度[m/s]
%   theta_g: 坡度角[rad], params: 参数结构体
% 输出参数:
%   N_lf, N_rf, N_lr, N_rr: 四轮垂向载荷[N]
%   F_rolling_total: 总滚动阻力[N]

% 提取参数
m = params.mass; % 质量[kg]
g = params.gravity; % 重力加速度[m/s²]
h_cg = params.h_cg; % 质心高度[m]
W = params.W; % 轮距[m]
L = params.L; % 轴距[m]
c_r = params.rolling_resistance; % 滚动阻力系数
accel_limit = params.max_acceleration; % 最大加速度限制[m/s²]

% 纵向加速度估算 (避免代数环)
F_rolling_est = c_r * m * g * cos(theta_g); % 估算滚动阻力[N]
F_aero_est = 0.5 * 1.225 * 0.5 * v^2 * sign(v); % 估算空气阻力[N] (简化)
F_slope_est = m * g * sin(theta_g); % 坡度阻力[N]
F_net_est = F_cmd - F_rolling_est - F_aero_est - F_slope_est; % 净驱动力[N]
a_long = sat(F_net_est/m, -accel_limit, accel_limit); % 纵向加速度[m/s²] (限幅)

% 纵向载荷转移计算
Delta_long = m * a_long * (h_cg/L); % 前后轴载荷转移量[N]

% 横向加速度估算 (基于当前运动状态)
% RF/LR 为万向支撑轮, 只参与垂向支撑; 侧向/偏航动力主要由 LF/RR 舵驱轮承担。
% 载荷转移需要左右转对称: 幅值取 |v*omega|, 方向单独由 sign(omega) 决定。
if abs(v) > 1e-3 && abs(omega) > 1e-6
    a_lat = abs(v * omega); % 横向向心加速度幅值[m/s²]
    Delta_lat = m * a_lat * (h_cg/max(W,1e-6)); % 横向载荷转移量[N]
else
    Delta_lat = 0; % 静止或极低速时无横向载荷转移
end

% 转向方向符号
sgn_turn = sign(omega);

% 基础轴载荷计算 (考虑坡度影响)
W_total = m * g * cos(theta_g); % 总垂向载荷 (坡度修正) [N]
N_front_base = W_total/2 - Delta_long/2; % 前轴基础载荷[N]
N_rear_base = W_total/2 + Delta_long/2; % 后轴基础载荷[N]

% 横向载荷分配到各轴
Delta_lat_front = sgn_turn * Delta_lat/2; % 前轴横向转移量[N]
Delta_lat_rear = sgn_turn * Delta_lat/2; % 后轴横向转移量[N]

```

```

% 四轮垂向载荷计算（确保非负）
N_lf = max(0, N_front_base/2 - Delta_lat_front/2); % 左前轮载荷[N]
N_rf = max(0, N_front_base/2 + Delta_lat_front/2); % 右前轮载荷[N]
N_lr = max(0, N_rear_base/2 - Delta_lat_rear/2); % 左后轮载荷[N]
N_rr = max(0, N_rear_base/2 + Delta_lat_rear/2); % 右后轮载荷[N]

% 基于实时载荷计算总滚动阻力
F_rolling_total = c_r * (N_lf + N_rf + N_lr + N_rr); % 总滚动阻力[N]
end

===== FILE: src/core/output_eq.m =====
function y = output_eq(x, u, theta_ground, params)
% =====
% 文件名: output_eq.m
% 功能描述: 对角式双舵轮 AGV 输出方程
% 输入参数:
%   x: 状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
%   u: 输入向量 [F_cmd omega_cmd] (2×1)
%   theta_ground: 地面坡度角（标量）
%   params: 参数结构体
% 输出参数:
%   y: 输出向量 (31×1) - V4.2 扩展版本
%   [1-8] 基本状态变量: X, Y, psi, v, omega, delta_lf, delta_rr, beta
%   [9-11] IMU 测量: accel_x_meas, gyro_y_meas, gyro_z_meas
%   [12-13] 电流估计: I_meas_lf, I_meas_rr
%   [14-15] 扰动估计: F_dist_calc_lf, F_dist_calc_rr
%   [16] 地面角度: theta_ground
%   [17-18] 轮速: omega_wheel_lf, omega_wheel_rr
%   [19-27] AI 算法专用输出: 载荷信息、轮胎利用率、运动状态（含前后侧偏角）
%   [28-31] 扩展诊断信息: slip_flag, stall_flag, N_lf, N_rr
% =====

%% 提取状态变量
X = x(1); Y = x(2); psi = x(3); v = x(4);
omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);

%% 提取输入
F_cmd = u(1);
omega_cmd = u(2);

%% 直接获取参数（从 parameters.m 结构体）
Ts = params.Ts;
m = params.mass;
I_z = params.Iz;
L = params.L;
W = params.W;
h_cg = params.h_cg;
r = params.wheel_radius;
n = params.gear_ratio;
eta = params.gear_efficiency;
k_t = params.motor_torque_constant;
mu = params.friction_coefficient;
c_r = params.rolling_resistance;
rho = params.air_density;
CdA = params.drag_coefficient_area;

```

```

C_af = params.front_cornering_stiffness;
C_ar = params.rear_cornering_stiffness;
delta_max = params.max_steering_angle;
delta_dot_max = params.max_steering_rate;
tau_delta = params.steering_time_constant;
current_limit = params.current_limit;
accel_limit = params.max_acceleration;
g = params.gravity;
min_omega_threshold = params.min_angular_velocity_threshold;
low_speed_thresh = params.low_speed_threshold;

% 噪声参数
enable_noise = params.enable_noise;
current_noise_std = params.current_noise_std;
wheel_speed_noise_std = params.wheel_speed_noise_std;
dist_noise_std = params.disturbance_noise_std;
noise_factor = double(enable_noise);

% 初始化随机数生成器（确保可复现，只在第一次调用时设置）
persistent seeded;
if enable_noise && params.random_seed > 0 && isempty(seeded)
    rng(params.random_seed);
    seeded = true;
end

%% === 几何：对角式转向目标（严格 ICR 约束，使用测量  $\omega$  抑制增益放大） ===
if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh
    delta_lf_target = 0; delta_rr_target = 0;
else
    % ICR 坐标（车辆坐标系）：用测量  $\omega$  替代命令  $\omega$ ，避免  $v \downarrow$  时目标舵角被放大
    w_icr = omega; % 使用测量值
    R_cmd = (v / max(abs(w_icr), 1e-6)) * sign(w_icr);
    x_c = 0.0; y_c = R_cmd;
    % 公式：tan(delta_lf) = (L/2 - x_c) / (y_c - W/2), tan(delta_rr) = -(x_c + L/2) / (y_c + W/2)
    delta_lf_target = atan2((L/2 - x_c), (y_c - W/2));
    delta_rr_target = -atan2((x_c + L/2), (y_c + W/2));
end

% NOTE(open-loop test): 观测侧不更新/不预测转角，仅读取状态中的
% delta_lf/delta_rr。原“舵机一阶”计算对输出无影响，已移除以避免误读。

%% === 载荷转移 & 滚阻 ===
[N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground, params);

%% === 电机/传动限制推算到车轮牵引上限 ===
F_wheel_max = current_limit * k_t * (eta * n) / max(r, 1e-6); % 单驱动轮
num_drive_wheels = 2; % lf & rr
F_cmd_max_total = num_drive_wheels * F_wheel_max;
F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);

%% === 侧偏角（简单自行车模型的近似，用  $\beta$  作为全车质心侧滑角） ===
% 轮侧偏角近似：alpha_f = delta_lf - (v_y + Lf*omega)/v_x ≈ delta_lf - (beta + Lf*omega/v)
% alpha_r = - (beta - Lr*omega/v)
Lf = L/2; Lr = L/2; % 对称
v_x = max(v * cos(beta), 1e-3);

```

```

alpha_f = (beta + Lf*omega/max(v,1e-3)) - delta_lf;
alpha_r = (beta - Lr*omega/max(v,1e-3)) - delta_rr; % 对角式双舵轮，后轮也有转向

% 线性侧偏刚度 + 软饱和（限幅到  $\mu N$ ）
Fy_f_lin = -C_af * alpha_f;
Fy_r_lin = -C_ar * alpha_r;
Fy_f_max = mu * N_lf;
Fy_r_max = mu * N_rr;
% 侧向力限幅（支持光滑限幅）
use_smooth = params.use_smooth_saturation;
if use_smooth
    smooth_gain = params.smooth_gain;
    Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
    Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
else
    Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
    Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
end

%% === 牵引力分配：按法向载荷比例 + 偏航控制 + 摩擦椭圆限幅 ===
% 修正：只在两个驱动轮（左前+右后）之间按载荷比例分配
W_drive = N_lf + N_rr; % 驱动轮总载荷（非全车总重）
w_lf = N_lf / max(W_drive, 1e-6); % 左前轮在驱动轮中的载荷占比
w_rr = N_rr / max(W_drive, 1e-6); % 右后轮在驱动轮中的载荷占比

% 直线/低曲率时等分扭矩，使观测与动力学一致
if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
    w_lf = 0.5;
    w_rr = 0.5;
end

% 恢复：观测侧与动力学保持一致的偏航力差估算（仅用于拆算分配，不闭环）
K_omega_p = 50.0;
e_omega = omega_cmd - omega;
omega_dot_desired = K_omega_p * e_omega;
omega_dot_max = 10.0;
omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
Mz_needed = I_z * omega_dot_desired;
Delta_Fx = 2 * Mz_needed / max(W, 1e-3);
Delta_Fx_max = max(abs(F_cmd_eff), 10) * 0.5;
Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);

% 驱动力分配：基础（载荷比例）+ 偏航控制（力差）
Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2;
Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2;

% 摩擦椭圆：sqrt((Fx/ $\mu N$ )^2 + (Fy/ $\mu N$ )^2) <= 1
Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf, 1e-6))^2, 0));
Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr, 1e-6))^2, 0));
F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);

% 合力/合力矩
F_drive_total = F_drive_lf + F_drive_rr;

```

```

% 空气阻力 + 滚动阻力 + 坡度分量
F_aero = 0.5 * rho * CdA * v^2 * sign(v);
F_drag = F_rolling_total + F_aero;
F_slope = m * g * sin(theta_ground);

% 有效质量（加入旋转惯量等效）
wheel_inertia = params.wheel_inertia;
motor_inertia = params.motor_inertia;
m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2);

%% === 计算测量量 ===

% [9-11] IMU 测量（添加噪声）
accel_x_base = (F_drive_total - F_drag - F_slope) / max(m_eff_total, 1e-6);
accel_x_meas = accel_x_base + noise_factor * 0.1 * randn;
gyro_y_meas = 0 + noise_factor * 0.01 * randn; % 俯仰角速度（添加小噪声）
gyro_z_meas = omega + noise_factor * 0.02 * randn;

% [17-18] 轮速（基于 ICR 几何/低速保护）
if abs(omega) > min_omega_threshold && abs(v) > 1e-4
    R = v / omega; sgn = sign(omega); x_c = 0; y_c = R*sgn;
    r_lf = sqrt((x_c - L/2)^2 + (y_c - W/2*sgn)^2);
    r_rr = sqrt((x_c + L/2)^2 + (y_c + W/2*sgn)^2);
    v_lf = (r_lf / max(abs(R), 1e-6)) * v;
    v_rr = (r_rr / max(abs(R), 1e-6)) * v;
else
    v_lf = v * cos(beta);
    v_rr = v * cos(beta);
end
omega_wheel_lf = v_lf / max(r, 1e-6) + noise_factor * wheel_speed_noise_std * randn;
omega_wheel_rr = v_rr / max(r, 1e-6) + noise_factor * wheel_speed_noise_std * randn;

% [12-13] 电流估计（车轮侧力矩映射回电机电流）
I_meas_lf = F_drive_lf * r / max(n*eta*k_t, 1e-6) + noise_factor * current_noise_std * randn;
I_meas_rr = F_drive_rr * r / max(n*eta*k_t, 1e-6) + noise_factor * current_noise_std * randn;

% [14-15] 扰动观测（净外力残差的简单估计）：
F_inertia_total = m_eff_total * accel_x_meas;
W_total = m*g*cos(theta_ground); % 补充定义 W_total（全车重量，用于扰动估计分配）
w_ratio_lf = N_lf / max(W_total, 1e-6);
w_ratio_rr = N_rr / max(W_total, 1e-6);
F_inertia_lf = F_inertia_total * w_ratio_lf; F_inertia_rr = F_inertia_total * w_ratio_rr;
F_slope_lf = F_slope * w_ratio_lf; F_slope_rr = F_slope * w_ratio_rr;
F_rolling_lf = c_r * N_lf; F_rolling_rr = c_r * N_rr;
F_motor_actual_lf = F_drive_lf; F_motor_actual_rr = F_drive_rr;
F_dist_calc_lf = F_motor_actual_lf - F_rolling_lf - F_inertia_lf - F_slope_lf +
noise_factor*dist_noise_std*randn;
F_dist_calc_rr = F_motor_actual_rr - F_rolling_rr - F_inertia_rr - F_slope_rr +
noise_factor*dist_noise_std*randn;

%% 计算额外的 AI 算法输出变量
% 载荷分配信息
load_ratio_front = (N_lf + N_rf) / max(W_total, 1e-6);
load_ratio_rear = (N_lr + N_rr) / max(W_total, 1e-6);
load_transfer_lateral = abs(N_lf - N_rf) / max(N_lf + N_rf, 1e-6);

% 轮胎利用率（摩擦椭圆）

```

```

tire_utilization_lf = sqrt((F_drive_lf/max(mu*N_lf,1e-6))^2 + (Fy_f/max(mu*N_lf,1e-6))^2);
tire_utilization_rr = sqrt((F_drive_rr/max(mu*N_rr,1e-6))^2 + (Fy_r/max(mu*N_rr,1e-6))^2);

% 运动状态指标
lateral_accel = v * omega; % 横向加速度估计 [m/s^2] - 修正量纲错误
slip_angle_front = alpha_f;
slip_angle_rear = alpha_r;

% 驱动状态
drive_force_asymmetry = abs(F_drive_lf - F_drive_rr) / max(F_drive_lf + F_drive_rr, 1e-6);

% 二元状态标志
slip_flag = double(max(tire_utilization_lf, tire_utilization_rr) > 0.98); % 打滑标志
total_current = I_meas_lf + I_meas_rr;
motor_current_limit_total = 2 * current_limit;
stall_flag = double(v < low_speed_thresh && total_current > 0.8 * motor_current_limit_total); % 堵转标志

%% 汇总输出 (31×1) - 为AI 算法提供丰富信息
y = [
    % [1-8] 基本状态变量
    X; Y; psi; v; omega; delta_lf; delta_rr; beta;
    % [9-11] IMU 测量
    accel_x_meas; gyro_y_meas; gyro_z_meas;
    % [12-13] 电流估计
    I_meas_lf; I_meas_rr;
    % [14-15] 驱动力估计
    F_dist_calc_lf; F_dist_calc_rr;
    % [16] 地面角度
    theta_ground;
    % [17-18] 轮速
    omega_wheel_lf; omega_wheel_rr;
    % [19-26] AI 算法专用输出
    load_ratio_front; load_ratio_rear; load_transfer_lateral; % 载荷信息
    tire_utilization_lf; tire_utilization_rr; % 轮胎利用率
    lateral_accel; slip_angle_front; slip_angle_rear; drive_force_asymmetry; % 运动状态
    % [28-31] 扩展诊断信息
    slip_flag; stall_flag; % 二元状态标志
    N_lf; N_rr % 法向载荷 (Fz)
];

end

%% ===== 工具函数 =====
function y = sat(x, xmin, xmax)
y = min(max(x, xmin), xmax);
end

function y = sat_sym(x, xmax)
y = min(max(x, -xmax), xmax);
end

function y = sat_smooth(x, xmin, xmax, k)
% 光滑限幅函数:  $\text{sat}(x,a,b) \approx a + (b-a) * (\tanh(k*(x-a)/(b-a))+1)/2$ 
% k: 光滑增益, 越大越接近硬限幅
if nargin < 4

```

```

    k = 30.0; % 默认增益
end
if xmax <= xmin
    y = (xmin + xmax) / 2;
    return;
end
x_norm = (x - xmin) / (xmax - xmin); % 归一化到[0,1]
y_norm = (tanh(k * (x_norm - 0.5)) + 1) / 2; % 光滑 S 曲线
y = xmin + (xmax - xmin) * y_norm;
end

function [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_g, params)
% 基于纵向/横向加速度的简化载荷转移 + 坡度影响
m = params.mass;
g = params.gravity;
h_cg = params.h_cg;
W = params.W;
L = params.L;
c_r = params.rolling_resistance;
accel_limit = params.max_acceleration;

% 纵向加速度（使用指令近似，以避免环依赖；也可改为用上一拍估计）
a_long = sat(F_cmd/m, -accel_limit, accel_limit) - g*sin(theta_g);
Delta_long = m * a_long * (h_cg/L); % 前后轴载荷转移

% 横向加速度（用 omega, v 估）
% RF/LR 为万向支撑轮，只参与垂向支撑；侧向/偏航动力主要由 LF/RR 舵驱轮承担。
% 载荷转移幅值取 |v*omega|，方向由 sign(omega) 单独决定，避免右转时被夹到近零。
if abs(v) > 1e-3 && abs(omega) > 1e-6
    a_lat = abs(v * omega);
    Delta_lat = m * a_lat * (h_cg/max(W,1e-6));
else
    Delta_lat = 0;
end
sgn_turn = sign(omega);

% 轴基础载荷（坡度影响投影到总重）
W_total = m * g * cos(theta_g);
N_front_base = W_total/2 - Delta_long/2;
N_rear_base = W_total/2 + Delta_long/2;

% 横向分配到轴
Delta_lat_front = sgn_turn * Delta_lat/2;
Delta_lat_rear = sgn_turn * Delta_lat/2;

% 四轮法向载荷
N_lf = max(0, N_front_base/2 - Delta_lat_front/2);
N_rf = max(0, N_front_base/2 + Delta_lat_front/2);
N_lr = max(0, N_rear_base/2 - Delta_lat_rear/2);
N_rr = max(0, N_rear_base/2 + Delta_lat_rear/2);

% 滚动阻力（基于实时载荷）
F_rolling_total = c_r * (N_lf + N_rf + N_lr + N_rr);
end

```

```

===== FILE: src/core/state_eq_ref.m =====
function x_next = state_eq_ref(x, u, theta_ground, params)
% =====
% 文件名: state_eq.m
% 功能描述: 对角式双舵轮 AGV 状态转移方程 - 实现车辆动力学模型的时间步进
% 输入参数:
%   x: 当前时刻状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
%       其中: X-全局 X 坐标[m], Y-全局 Y 坐标[m], psi-航向角[rad]
%             v-纵向速度[m/s], omega-横摆角速度[rad/s]
%             delta_lf-左前轮转向角[rad], delta_rr-右后轮转向角[rad]
%             beta-质心侧偏角[rad]
%   u: 控制输入向量 [F_cmd omega_cmd] (2×1)
%       F_cmd-总驱动力指令[N], omega_cmd-期望横摆角速度[rad/s]
%   theta_ground: 地面坡度角[rad] (标量)
%   params: 包含所有车辆参数的结构体
% 输出参数:
%   x_next: 下一时刻状态向量 (8×1)
% =====

%% 提取状态变量 - 从状态向量中分解各个状态分量
X = x(1); Y = x(2); psi = x(3); v = x(4);
omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);

%% 提取控制输入 - 从输入向量分解控制指令
F_cmd = u(1); % 总驱动力指令[N]
omega_cmd = u(2); % 期望横摆角速度指令[rad/s]

%% 直接从参数结构体获取车辆参数 - 提高代码可读性和维护性
Ts = params.Ts; % 采样时间[s]
m = params.mass; % 车辆总质量[kg]
I_z = params.Iz; % 绕 Z 轴的转动惯量[kg·m²]
L = params.L; % 轴距[m]
W = params.W; % 轮距[m]
h_cg = params.h_cg; % 质心高度[m]
r = params.wheel_radius; % 车轮半径[m]
n = params.gear_ratio; % 传动比
eta = params.gear_efficiency; % 传动效率
k_t = params.motor_torque_constant; % 电机转矩常数[N·m/A]
mu = params.friction_coefficient; % 路面摩擦系数
c_r = params.rolling_resistance; % 滚动阻力系数
rho = params.air_density; % 空气密度[kg/m³]
CdA = params.drag_coefficient_area; % 风阻系数×迎风面积[m²]
C_af = params.front_cornering_stiffness; % 前轮侧偏刚度[N/rad]
C_ar = params.rear_cornering_stiffness; % 后轮侧偏刚度[N/rad]
delta_max = params.max_steering_angle; % 最大转向角限制[rad]
delta_dot_max = params.max_steering_rate; % 最大转向速率[rad/s]
tau_delta = params.steering_time_constant; % 转向系统时间常数[s]
current_limit = params.current_limit; % 电机电流限制[A]
accel_limit = params.max_acceleration; % 最大加速度限制[m/s²]
g = params.gravity; % 重力加速度[m/s²]
min_omega_threshold = params.min_angular_velocity_threshold; % 最小角速度阈值[rad/s]
low_speed_thresh = params.low_speed_threshold; % 低速阈值[m/s]

%% === 转向几何计算: 参考几何 ICR (|ω_ref|<阈值回退测量) ===
if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh

```



```

% 低速/小指令：维持直行
delta_lf_target = 0; delta_rr_target = 0;
else
% 角速度误差抑制（比例缩放，保持两轮几何比不变）
e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
if abs(omega) > abs(omega_cmd) * 1.2 && sign(omega) == sign(omega_cmd)
    delta_scale = max(omega_cmd / max(abs(omega), 1e-6), 0.1);
else
    delta_scale = 1.0;
end

% ICR：优先用参考曲率 R_ref=v_ref/omega_ref; |w_ref|<阈值或缺失时回退测量
w_ref_min = 1e-3; % 回退阈值 [rad/s]
try, ref = evalin('base','ref'); catch, ref = struct(); end
v_ref_use = NaN; omega_ref_use = NaN;
if isstruct(ref) && isfield(ref,'v_ref') && isfield(ref,'omega_ref')
    if isfield(ref,'t') && ~isempty(ref.t)
        try, t_sim = get_param(bdroot,'SimulationTime'); catch, t_sim = 0; end
        Ts_loc = Ts;
        idx = 1 + round(t_sim / max(Ts_loc,1e-6));
        idx = max(1, min(idx, numel(ref.t)));
        v_ref_use = ref.v_ref(idx);
        omega_ref_use = ref.omega_ref(idx);
    else
        if isscalar(ref.v_ref), v_ref_use = ref.v_ref; end
        if isscalar(ref.omega_ref), omega_ref_use = ref.omega_ref; end
    end
end
use_ref = ~isnan(omega_ref_use) && abs(omega_ref_use) >= w_ref_min && ~isnan(v_ref_use);
if use_ref
    R_cmd = (v_ref_use / max(abs(omega_ref_use),1e-6)) * sign(omega_ref_use);
else
    w_icr = omega; % 回退：使用测量 w
    R_cmd = (v / max(abs(w_icr),1e-6)) * sign(w_icr);
end
x_c = 0.0; % 对称转弯，ICR 在 y 轴上
y_c = R_cmd; % ICR y 坐标（可正可负）

% 使用带符号保护的分子，避免右转时象限翻转，同时保持右轮负号
denom_lf = y_c - W/2;
denom_rr = y_c + W/2;
safe_denom_lf = sign(denom_lf) * max(abs(denom_lf), 1e-6);
safe_denom_rr = sign(denom_rr) * max(abs(denom_rr), 1e-6);
delta_lf_geom = atan((L/2 - x_c) / safe_denom_lf);
delta_rr_geom = -atan((x_c + L/2) / safe_denom_rr);

% 误差缩放（不破坏两轮几何比例）
delta_lf_target = delta_lf_geom * delta_scale;
delta_rr_target = delta_rr_geom * delta_scale;

% 取消额外的“安全夹紧”：由执行器限幅 delta_max 与速率限幅保障
end

%% === 舵机动力学模拟：一阶惯性 + 速率限制 + 角度限制 ===
% 转向系统动力学：一阶惯性环节模拟转向执行器响应

```

```

delta_lf_dot = (delta_lf_target - delta_lf)/max(tau_delta,Ts);
delta_rr_dot = (delta_rr_target - delta_rr)/max(tau_delta,Ts);

% 转向速率限制 - 模拟转向执行器的物理限制
delta_lf_dot = sat(delta_lf_dot, -delta_dot_max, delta_dot_max);
delta_rr_dot = sat(delta_rr_dot, -delta_dot_max, delta_dot_max);

% 转向角更新 + 角度限制
delta_lf_new = sat(delta_lf + Ts*delta_lf_dot, -delta_max, delta_max);
delta_rr_new = sat(delta_rr + Ts*delta_rr_dot, -delta_max, delta_max);

%% === 载荷转移计算 & 滚动阻力计算 ===
[N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground, params);

%% === 电机/传动系统限制推算到车轮牵引力上限 ===
% 计算单个驱动轮的最大牵引力（基于电机电流限制）
F_wheel_max = current_limit * k_t * (eta * n) / max(r,1e-6); % 单驱动轮最大牵引力[N]

% 计算总驱动轮的最大牵引力（对角式：左前+右后两个驱动轮）
num_drive_wheels = 2; % 驱动轮数量: lf & rr
F_cmd_max_total = num_drive_wheels * F_wheel_max;

% 应用牵引力限制到指令值
F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);

%% === 轮胎侧偏角计算（基于简化自行车模型） ===
% 车辆几何参数：假设质心位于轴距中心
Lf = L/2; Lr = L/2; % 前后轴到质心的距离[m]

% 侧偏刚度：使用标称值（取消转弯增强的临时放大）
% C_af, C_ar 已由 parameters.m 提供

% 纵向速度分量（防止除零）
v_x = max(v * cos(beta), 1e-3);

% 前轮侧偏角（标准符号）: alpha_f = (beta + Lf*omega/v) - delta_lf
alpha_f = (beta + Lf*omega/max(v,low_speed_thresh)) - delta_lf_new;

% 后轮侧偏角（标准符号）: alpha_r = (beta - Lr*omega/v) - delta_rr
alpha_r = (beta - Lr*omega/max(v,low_speed_thresh)) - delta_rr_new; % 对角式双舵轮，后轮也有转向

% 线性侧偏力计算 + 软饱和（限幅到摩擦圆边界）
Fy_f_lin = -C_af * alpha_f; % 前轮线性侧偏力[N]
Fy_r_lin = -C_ar * alpha_r; % 后轮线性侧偏力[N]

% 基于垂向载荷计算最大侧向力（摩擦圆限制）
Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]

% 应用侧向力饱和和限制
Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);

%% === 牵引力分配策略：按法向载荷比例 + 偏航控制 ===
% 驱动轮总垂向载荷（仅考虑两个驱动轮：左前+右后）

```

```

W_drive = N_lf + N_rr; % 驱动轮总载荷[N]

% 计算各驱动轮在总驱动载荷中的权重比例
w_lf = N_lf / max(W_drive, 1e-6); % 左前轮载荷占比
w_rr = N_rr / max(W_drive, 1e-6); % 右后轮载荷占比

% 直线/低曲率时等分扭矩，抑制对角驱动导致的偏航力矩
if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
    w_lf = 0.5;
    w_rr = 0.5;
end

% 偏航控制：根据角速度误差产生驱动力差以实现横摆控制
e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]

% 横摆角速度保护：防止角速度爆炸式增长
omega_max = 2.0; % 最大角速度限制 [rad/s] (远大于目标 0.1 rad/s)

if v < low_speed_thresh || abs(omega_cmd) < min_omega_threshold
    % 低速/小指令：关断偏航力差，避免原地自旋
    Delta_Fx = 0;
else
    if abs(omega) > omega_max
        % 情况：角速度超限 - 施加强制制动力矩进行保护
        omega_sign = sign(omega);
        Mz_brake = -omega_sign * I_z * 50.0; % 强制制动力矩[Nm]
        Delta_Fx = 2 * Mz_brake / max(W, 1e-3); % 所需的驱动力差[N]
    else
        % 恢复：正常 P 控制产生偏航力差
        K_omega_p = 120.0; % 偏航控制增益 (优化: 100→120)
        omega_dot_desired = K_omega_p * e_omega; % 期望角加速度[rad/s²]
        omega_dot_max = 10.0; % 最大角加速度 [rad/s²]
        omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
        Mz_needed = I_z * omega_dot_desired; % 所需偏航力矩 [Nm]
        Delta_Fx = 2 * Mz_needed / max(W, 1e-3); % 所需驱动力差 [N]
        % 解耦：按摩擦裕度设置 ΔFx_max (不再绑定 F_cmd_eff 大小)
        Delta_Fx_max = 0.5 * mu * (N_lf + N_rr); % 驱动力差限幅 (保守系数 0.5)
        Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);
    end
end

% 驱动力分配：基础分配 (按载荷比例) + 偏航控制 (力差调整)
Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2; % 左前轮指令驱动力[N]
Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2; % 右后轮指令驱动力[N]

%% === 摩擦椭圆限幅：确保合力不超过轮胎-路面摩擦极限 ===
% 获取是否使用光滑限幅的标志
use_smooth = params.use_smooth_saturation;

if use_smooth
    % 光滑版本限幅：使用 softplus 函数替代 max(·, 0)，提高数值稳定性
    smooth_gain = params.smooth_gain;

    % 计算各轮胎在摩擦椭圆中的可用纵向力余量
    term_lf = softplus(1 - (Fy_f/max(mu*N_lf, 1e-6))^2, smooth_gain);

```

```

term_rr = softplus(1 - (Fy_r/max(mu*N_rr,1e-6))^2, smooth_gain);

Fx_lf_allow = mu * N_lf * sqrt(term_lf); % 左前轮允许的最大纵向力[N]
Fx_rr_allow = mu * N_rr * sqrt(term_rr); % 右后轮允许的最大纵向力[N]

% 应用光滑限幅到指令驱动力
F_drive_lf = sat_smooth(Fx_lf_cmd, -Fx_lf_allow, Fx_lf_allow, smooth_gain);
F_drive_rr = sat_smooth(Fx_rr_cmd, -Fx_rr_allow, Fx_rr_allow, smooth_gain);
else
% 硬限幅版本: 传统的摩擦椭圆限制
Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf,1e-6))^2, 0));
Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr,1e-6))^2, 0));

F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);
end

% 计算总有效驱动力
F_drive_total = F_drive_lf + F_drive_rr;

% 计算驱动不对称产生的偏航力矩 (左右驱动力差异引起)
Mz_drive = (W/2) * (F_drive_lf - F_drive_rr);

% 坡度阻力计算
F_slope = m * g * sin(theta_ground);

% 有效质量计算 (包含旋转部件的等效质量)
wheel_inertia = params.wheel_inertia; % 车轮转动惯量[kg·m²]
motor_inertia = params.motor_inertia; % 电机转动惯量[kg·m²]
m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2); % 总有效质量[kg]

%% === RK4 数值积分 (转向角线性插值 + 动态空气阻力计算) ===
% 提取核心状态变量用于 RK4 积分
s_k = [X; Y; psi; v; omega; beta];

% 转向角中点值 (用于 RK4 中间步骤的线性插值)
delta_lf_mid = (delta_lf + delta_lf_new)/2;
delta_rr_mid = (delta_rr + delta_rr_new)/2;

% 获取光滑限幅参数
use_smooth = params.use_smooth_saturation;
smooth_gain = params.smooth_gain;

% 定义连续动力学核心函数 (闭包形式, 传入当前计算的所有参数)
CONT_CORE = @(s, dl, dr) continuous_dynamics_core_v2(s, dl, dr, ...
    F_drive_lf, F_drive_rr, F_rolling_total, F_slope, m_eff_total, I_z, Lf, Lr, ...
    low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
    rho, CdA); % 传入空气阻力参数用于动态计算

% RK4 积分四步计算 (使用转向角插值)
k1 = CONT_CORE(s_k, delta_lf, delta_rr); % 第一步: 使用 k 时刻转向角
k2 = CONT_CORE(s_k + 0.5*Ts*k1, delta_lf_mid, delta_rr_mid); % 第二步: 使用中点转向角
k3 = CONT_CORE(s_k + 0.5*Ts*k2, delta_lf_mid, delta_rr_mid); % 第三步: 使用中点转向角
k4 = CONT_CORE(s_k + Ts*k3, delta_lf_new, delta_rr_new); % 第四步: 使用 k+1 时刻转向角

```

```

% RK4 最终积分：加权平均
s_new = s_k + (Ts/6)*(k1 + 2*k2 + 2*k3 + k4);

%% 提取新状态并应用保护措施
X_new = s_new(1); Y_new = s_new(2); psi_new = s_new(3);
v_new = s_new(4); omega_new = s_new(5); beta_new = s_new(6);

% 航向角归一化到[-pi, pi]范围
psi_new = normalizeAngle(psi_new);

% 低速保护：防止速度变为负值（车辆不能倒车）
if v_new < 0
    v_new = 0;
end

% 侧偏角限幅保护（防止数值爆炸）
beta_new = sat(beta_new, -deg2rad(15), deg2rad(15));

%% 组装下一时刻完整状态向量
x_next = [X_new; Y_new; psi_new; v_new; omega_new; delta_lf_new; delta_rr_new; beta_new];

end

%% ===== 连续动力学核心函数 V2（动态空气阻力计算） =====
function ds = continuous_dynamics_core_v2(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_rolling_total, F_slope, ...
    m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
    rho, CdA)
% 连续动力学核心函数 V2 - 用于 RK4 积分的每个子步，动态计算空气阻力
% 输入参数：
%   s: 状态向量 [X Y psi v omega beta]
%   delta_lf, delta_rr: 当前转向角[rad]
%   Fx_lf, Fx_rr: 预计算的驱动力（已应用摩擦椭圆限幅）[N]
%   F_rolling_total: 总滚动阻力[N]
%   F_slope: 坡度阻力[N]
%   m_eff_total: 总有效质量[kg]
%   I_z: 转动惯量[kg·m²]
%   Lf, Lr: 前后轴到质心距离[m]
%   low_speed_thresh: 低速阈值[m/s]
%   C_af, C_ar: 前后轮侧偏刚度[N/rad]
%   mu: 摩擦系数
%   N_lf, N_rr: 左右驱动轮垂向载荷[N]
%   W: 轮距[m]
%   use_smooth: 是否使用光滑限幅
%   smooth_gain: 光滑限幅增益
%   rho: 空气密度[kg/m³]
%   CdA: 风阻系数×面积[m²]
% 输出参数：
%   ds: 状态导数向量 [X_dot Y_dot psi_dot v_dot omega_dot beta_dot]

% 设置默认参数值
if nargin < 22
    use_smooth = false;
    smooth_gain = 30.0;
end

```

```

% 分解状态向量
X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);

% 动态计算空气阻力（基于当前瞬时速度）
F_aero = 0.5 * rho * CdA * v^2 * sign(v); % 空气阻力[N], 与速度平方成正比
F_drag = F_rolling_total + F_aero; % 总阻力 = 滚动阻力 + 空气阻力

% 重新计算轮胎侧偏角（基于当前状态和转向角）
alpha_f = (beta + Lf*omega/max(abs(v),low_speed_thresh)) - delta_lf;
alpha_r = (beta - Lr*omega/max(abs(v),low_speed_thresh)) - delta_rr;

% 线性侧偏力计算
Fy_f_lin = -C_af * alpha_f; % 前轮侧偏力[N]
Fy_r_lin = -C_ar * alpha_r; % 后轮侧偏力[N]

% 侧向力饱和限制（基于摩擦圆原理）
Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]

% 应用侧向力限幅（可选择光滑或硬限幅）
if use_smooth
    Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
    Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
else
    Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
    Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
end

% 使用预计算的驱动力（避免重复限幅）
F_drive_actual = Fx_lf + Fx_rr; % 实际总驱动力[N]

% 驱动不对称产生的偏航力矩
Mz_drive = (W/2) * (Fx_lf - Fx_rr); % [Nm]

% 纵向动力学方程
a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total,1e-6); % 纵向加速度[m/s²]

% 低速保护：防止倒车情况下的负加速度
if a_x < 0 && v < 0
    v_dot = 0; % 速度为负时不允许进一步减速
else
    v_dot = a_x; % 正常加速度
end

% 横摆动力学计算
Mz_tire = Fy_f*Lf - Fy_r*Lr; % 轮胎侧偏力产生的偏航力矩[Nm]
Mz_total = Mz_tire + Mz_drive; % 总偏航力矩 = 轮胎力矩 + 驱动力矩

% 调整：横摆阻尼减小以提高可转性
C_damping = 250.0; % 横摆阻尼系数 [Nm/(rad/s)] (优化: 400→250)
Mz_damping = -C_damping * omega; % 阻尼力矩（与角速度成正比）[Nm]
Mz_total = Mz_total + Mz_damping; % 包含阻尼的总偏航力矩[Nm]

% 角加速度计算
omega_dot = Mz_total / max(I_z,1e-6); % [rad/s²]

```

end

end

%% =====

function metrics = classification_metrics(y_true, y_pred, class_names)

% 统一计算准确率、精确率、召回率、F1 以及混淆矩阵

cm = confusionmat(y_true(:), y_pred(:));

num_classes = size(cm,1);

precision = zeros(num_classes,1);

recall = zeros(num_classes,1);

f1 = zeros(num_classes,1);

for c = 1:num_classes

tp = cm(c,c);

fp = sum(cm(:,c)) - tp;

fn = sum(cm(c,:)) - tp;

precision(c) = tp / max(tp + fp, eps);

recall(c) = tp / max(tp + fn, eps);

f1(c) = 2 * precision(c) * recall(c) / max(precision(c) + recall(c), eps);

end

metrics = struct();

metrics.accuracy = sum(diag(cm)) / sum(cm(:));

metrics.confusion = cm;

metrics.precision = precision;

metrics.recall = recall;

metrics.f1 = f1;

metrics.class_names = class_names(:)';

end

%% =====

function summary = build_summary(split_reports, online_reports, online_by_mode, seq_len)

% 聚合离线/在线指标，用于最终对比展示

summary = struct();

summary.seq_len = seq_len;

summary.offline = split_reports;

summary.online = online_reports;

summary.online_by_mode = online_by_mode;

summary.mean_acc_main = mean(cellfun(@(r) r.metrics_main.accuracy, split_reports));

summary.mean_mae_theta = mean(cellfun(@(r) r.mae_theta, split_reports));

if isempty(online_reports)

summary.mean_online_delay = NaN;

else

summary.mean_online_delay = mean(cellfun(@(r) r.slope_delay, online_reports), 'omitnan');

end

end

%% =====

function print_summary(summary)

% 终端输出核心指标，方便快速查看

fprintf(' - 平均主分类准确率 (Train/Val/Test) : %.2f%%\n', summary.mean_acc_main*100);

fprintf(' - 平均离线坡度 MAE: %.3f deg\n', rad2deg(summary.mean_mae_theta));

if isnan(summary.mean_online_delay)

fprintf(' - 平均在线坡度识别延迟: NaN (本次未选择到含坡度的 run) (seq_len=%.0f)\n', summary.seq_len);

else

fprintf(' - 平均在线坡度识别延迟: %.2f s (seq_len=%.0f)\n', summary.mean_online_delay, summary.seq_len);

```

end
print_mode_breakdown(summary.online_by_mode);
end

%% =====
function print_mode_breakdown(mode_entries)
% 输出按行驶模式分组的在线评估结果
if isempty(mode_entries)
    fprintf(' - 未选择任何行驶模式样本, 跳过在线分组统计.\n');
    return;
end

fprintf('\n 行驶模式分组指标: \n');
for i = 1:numel(mode_entries)
    reports_cell = mode_entries(i).reports;
    if isempty(reports_cell)
        continue;
    end
    reports = [reports_cell{:}];
    acc_main = mean(arrayfun(@(r) r.acc_main, reports));
    acc_turn = mean(arrayfun(@(r) r.acc_turn, reports));
    mae_theta = mean(arrayfun(@(r) r.mae_theta, reports), 'omitnan');
    delay = mean(arrayfun(@(r) r.slope_delay, reports), 'omitnan');
    fprintf(' - %-15s | 样本:%2d | 主分类:%.2f%% | 转弯:%.2f%% |  $\theta$  MAE:%.3f deg | 延迟:%s\n', ...
        mode_entries(i).mode, numel(reports), acc_main*100, acc_turn*100, rad2deg(mae_theta),
        format_delay(delay));
end
end

function txt = format_delay(delay)
if isnan(delay)
    txt = 'NaN';
else
    txt = sprintf('%.2f s', delay);
end
end

%% =====
function name = get_scene_name(source)
% 从 run/元数据中解析场景名称, 兼容嵌套结构与不同字段命名
default_name = 'run_unknown';
name = default_name;

if nargin == 0 || isempty(source)
    return;
end

% 直接传入字符串时立即返回
direct_text = extract_scene_text(source);
if ~isempty(direct_text)
    name = direct_text;
    return;
end

if ~isstruct(source)
    return;
end

```



```

end

text_fields = {'scene', 'path_type', 'scene_name', 'mode'};
for i = 1:numel(text_fields)
    field = text_fields{i};
    if isfield(source, field)
        candidate = extract_scene_text(source.(field));
        if ~isempty(candidate)
            name = candidate;
            return;
        end
    end
end

end

% 递归查找常见嵌套字段
nested_fields = {'meta', 'path_params'};
for i = 1:numel(nested_fields)
    field = nested_fields{i};
    if isfield(source, field)
        candidate = get_scene_name(source.(field));
        if ~strcmp(candidate, default_name)
            name = candidate;
            return;
        end
    end
end

end

end

function text = extract_scene_text(value)
% 统一提取字符串表示, 兼容 char/string/cellstr
text = '';
if ischar(value)
    text = strtrim(value);
elseif isstring(value) && isscalar(value)
    text = strtrim(char(value));
elseif iscell(value) && numel(value) == 1
    text = extract_scene_text(value{1});
end

if isstring(text)
    text = char(text);
end

if isempty(text)
    text = '';
end

end

%% =====
function cfg = apply_defaults(cfg, defaults)
% 递归填充缺省配置项, 保持主逻辑简洁
fields = fieldnames(defaults);
for i = 1:numel(fields)
    f = fields{i};

```

```

    if ~isfield(cfg, f) || isempty(cfg.(f))
        cfg.(f) = defaults.(f);
    end
end
end

===== FILE: src/tests/test_gru_latency.m =====
% =====
% 文件名: test_gru_latency.m
% 版本号: V1.1
% 最后修改时间: 2025-11-26
% 作者: LPV-MPC Project
% 功能描述:
%   使用控制变量法 + 预设组合验证, 对 GRU 在线识别中的
%   dwell_main / dwell_turn / tau_theta 三个延时敏感参数进行搜索,
%   输出综合评分排名与最优配置。
%
% 配置说明:
%   - cfg.model_file       : GRU 模型文件 (默认 GRU_model.mat)
%   - cfg.raw_data_file    : 原始 run 数据 (默认 GRU_train_data_full.mat)
%   - cfg.results_dir      : 评估结果输出目录
%   - cfg.runs_per_mode    : 每种行驶模式抽取的 run 数量
%   - cfg.weights          : 综合得分权重 (准确率/延时/θMAE/抖动)
%
% 依赖脚本/数据:
%   - parameters.m (系统参数)
%   - GRU_model.mat (包含 seq_len/scaler 的训练模型)
%   - GRU_train_data_full.mat (在线评估所需 run)
%   - GRU_state_classifier.m / GRU_infer.m (在线推理封装)
%
% 使用方法:
%   >> test_gru_latency
%   运行结束后, Top5 排行及控制变量法最优结果将打印至命令行,
%   并保存至 cfg.results_dir/latency_summary_<timestamp>.mat。
%
% 约束/注意:
%   - 仅覆盖 GRU_state_classifier 中的驻留时间与 theta 低通参数;
%   - seq_len、特征滤波 (tau_diff/tau_accel_lp) 与数据集保持不变;
%   - 若需评估其它参数, 应另行构建专用脚本。
% =====

clear; clc; % 清理环境, 以免旧变量干扰评分

root = project_root();
data_models_dir = fullfile(root, 'data', 'models');
data_gru_dir = fullfile(root, 'data', 'gru');
default_results_dir = results_dir('gru/latency_eval');
fprintf('\n===== \n');
fprintf('GRU 延时评估 (test_gru_latency)\n');
fprintf('===== \n');

%% 配置
cfg = struct();
cfg.model_file = fullfile(data_models_dir, 'GRU_model.mat');
cfg.raw_data_file = fullfile(data_gru_dir, 'GRU_train_data_full.mat');

```

```

cfg.results_dir      = default_results_dir;
cfg.runs_per_mode    = 6;           % 每种行驶模式最多选取的 run 数
cfg.weights = struct( ...
    'acc_main', 1.0, ...           % 主分类准确率权重
    'acc_turn', 0.5, ...           % 转弯准确率权重
    'delay',    0.7, ...           % 延时惩罚系数 (单位 s)
    'theta',    0.2, ...           %  $\theta$  MAE 惩罚系数 (单位 deg)
    'flips',    0.3 ...           % 标签抖动惩罚系数 (次/分钟)
);

if ~exist(cfg.results_dir,'dir')
    mkdir(cfg.results_dir);
end

%% 加载依赖与原始数据
fprintf('[1/6] 加载模型与数据...\n');
params = parameters();
model_data = load(cfg.model_file, 'model');
model = model_data.model;
raw = load(cfg.raw_data_file, 'data');
all_runs = raw.data.runs;
[selected_runs, selection_meta] = select_runs_for_latency(all_runs, cfg.runs_per_mode);
if isempty(selected_runs)
    error('未选到任何 run, 请检查原始数据。');
end
fprintf(' - 选取 %d 条 run, 覆盖 %d 种行驶模式\n', numel(selected_runs), selection_meta.num_modes);

timestamp = datestr(now,'yyyymmdd_HHMMSS');
variant_cache = containers.Map(); % key -> metrics
all_metrics = containers.Map();

%% 变体定义 (包含控制变量序列与人工组合)
% 最优组合 (来自上一轮评估):
% dwell_main = 0.20 s
% dwell_turn = 0.40 s
% tau_theta = 0.15 s
baseline = struct('name','baseline','dwell_main',0.20,'dwell_turn',0.40,'tau_theta',0.15);
dwell_main_candidates = [0.20 0.25 0.30 0.40];
dwell_turn_candidates = [0.40 0.50];
tau_theta_candidates = [0.15 0.20 0.40];
manual_combos = [ ...
    struct('name','combo_fast','dwell_main',0.25,'dwell_turn',0.40,'tau_theta',0.20); ...
    struct('name','combo_balanced','dwell_main',0.30,'dwell_turn',0.50,'tau_theta',0.20); ...
    struct('name','combo_aggressive','dwell_main',0.20,'dwell_turn',0.40,'tau_theta',0.15) ...
];

%% 基线评估: 记录默认参数的表现
fprintf('\n[2/6] 评估基线参数...\n');
[metrics, variant_cache] = eval_variant_with_cache(baseline, params, model, selected_runs, cfg, variant_cache);
all_metrics(metrics.key) = metrics;
control_variant = baseline;
control_metrics = metrics;
fprintf(' baseline score = %.3f (AccMain=%.3f, Delay=%.3f s)\n', control_metrics.score, control_metrics.acc_main, control_metrics.delay_combined);

%% 控制变量法: 仅改变 dwell_main, 锁定其余参数

```

```

fprintf('\n[3/6] 控制变量法 - 扫描 dwell_main ...\n');
[control_variant, control_metrics, variant_cache, all_metrics] = sweep_parameter('dwell_main', ...
    dwell_main_candidates, control_variant, params, model, selected_runs, cfg, variant_cache, all_metrics);
fprintf(' best dwell_main=%.2f s (score=%.3f)\n', control_variant.dwell_main, control_metrics.score);

%% 控制变量法: 仅改变 dwell_turn
fprintf('\n[4/6] 控制变量法 - 扫描 dwell_turn ...\n');
[control_variant, control_metrics, variant_cache, all_metrics] = sweep_parameter('dwell_turn', ...
    dwell_turn_candidates, control_variant, params, model, selected_runs, cfg, variant_cache, all_metrics);
fprintf(' best dwell_turn=%.2f s (score=%.3f)\n', control_variant.dwell_turn, control_metrics.score);

%% 控制变量法: 仅改变 tau_theta
fprintf('\n[5/6] 控制变量法 - 扫描 tau_theta ...\n');
[control_variant, control_metrics, variant_cache, all_metrics] = sweep_parameter('tau_theta', ...
    tau_theta_candidates, control_variant, params, model, selected_runs, cfg, variant_cache, all_metrics);
fprintf(' best tau_theta=%.2f s (score=%.3f)\n', control_variant.tau_theta, control_metrics.score);
control_best = control_metrics;

%% 手工组合验证: 捕捉潜在参数耦合
fprintf('\n[6/6] 评估手工组合...\n');
for i = 1:numel(manual_combos)
    combo = manual_combos(i);
    [combo_metrics, variant_cache] = eval_variant_with_cache(combo, params, model, selected_runs, cfg,
variant_cache);
    combo_metrics.tag = sprintf('manual:%s', combo.name);
    all_metrics(combo_metrics.key) = combo_metrics;
end

%% 汇总排行榜并输出 Top5
scoreboard = collect_metrics(all_metrics);
[~, order] = sort([scoreboard.score], 'descend');
scoreboard = scoreboard(order);

fprintf('\n=====');
fprintf('评估完成, Top 5 方案如下 (score 越大越好): \n');
fprintf('%-16s %-7s %-7s %-7s | Score AccMain AccTurn Delay 0MAE Flips/min\n', 'Variant', 'd_main',
'd_turn', 'tau0');
fprintf('%s\n', repmat('-',1,86));
for i = 1:min(5, numel(scoreboard))
    m = scoreboard(i);
    fprintf('%-16s %-7.2f %-7.2f %-7.2f | %5.3f %5.3f %5.3f %5.3f %5.3f %5.2f\n', ...
        m.tag, m.variant.dwell_main, m.variant.dwell_turn, m.variant.tau_theta, ...
        m.score, m.acc_main, m.acc_turn, m.delay_combined, m.theta_mae_deg, m.flip_rate_per_min);
end

fprintf('\n 控制变量法最优: %s (score=%.3f)\n', control_best.tag, control_best.score);
fprintf(' - dwell_main=%.2f s, dwell_turn=%.2f s, tau_theta=%.2f s\n', ...
    control_best.variant.dwell_main, control_best.variant.dwell_turn, control_best.variant.tau_theta);

summary_file = fullfile(cfg.results_dir, sprintf('latency_summary_%s.mat', timestamp));
save(summary_file, 'scoreboard', 'cfg', 'selection_meta', 'control_best');
fprintf('\n 已保存评估结果至 %s\n', summary_file);

%% 辅助函数
function [metrics, cache] = eval_variant_with_cache(variant, params, model, runs, cfg, cache)
    % 带缓存的变体评估, 避免重复仿真相同组合
    key = variant_key(variant);

```

```

    if isKey(cache, key)
        metrics = cache(key);
        return;
    end
    run_stats = arrayfun(@(r) simulate_and_score_run(r, params, model, variant), runs);
    metrics = aggregate_metrics(run_stats, variant, cfg);
    metrics.key = key;
    metrics.tag = variant.name;
    cache(key) = metrics;
end

function [best_variant, best_metrics, cache, metrics_map] = sweep_parameter(param_name, values,
base_variant, params, model, runs, cfg, cache, metrics_map)
    % 对单个参数做一维扫描，返回得分最高的组合
    best_variant = base_variant;
    best_metrics = [];
    best_score = -inf;
    for v = values
        variant = base_variant;
        variant.(param_name) = v;
        variant.name = sprintf('%s_%.2f', base_variant.name, param_name, v);
        [metrics, cache] = eval_variant_with_cache(variant, params, model, runs, cfg, cache);
        metrics.tag = sprintf('sweep:%s', param_name);
        metrics_map(metrics.key) = metrics;
        if metrics.score > best_score
            best_score = metrics.score;
            best_variant = variant;
            best_metrics = metrics;
        end
    end
end

function stats = simulate_and_score_run(run, params, model, variant)
    % 针对单条 run 调用在线推理，采集预测轨迹并计算指标
    state = GRU_state_classifier('init', params, model);
    state = apply_variant_overrides(state, variant);
    seq_len_effective = state.seq_len;
    N = size(run.y_raw,1);
    pred_main = zeros(N,1);
    pred_turn = zeros(N,1);
    pred_theta = zeros(N,1);
    for k = 1:N
        y = run.y_raw(k,:);
        [state, out] = GRU_state_classifier('update', state, y);
        pred_main(k) = out.label_main;
        pred_turn(k) = out.label_turn;
        pred_theta(k) = out.theta_hat;
    end
    stats.seq_len = seq_len_effective;
    stats.time = run.t;
    stats.truth_main = run.label_main;
    stats.truth_turn = run.label_turn;
    stats.truth_theta = run.theta;
    stats.pred_main = pred_main;
    stats.pred_turn = pred_turn;
    stats.pred_theta = pred_theta;
end

```

```

    stats = compute_run_metrics(stats);
end

function state = apply_variant_overrides(state, variant)
    % 将变体中的 dwell_main/dwell_turn/tau_theta 覆盖到 state
    if isfield(variant, 'dwell_main')
        state.dwell_main = variant.dwell_main;
        state.dwell_main_steps = max(1, round(state.dwell_main / state.Ts));
    end
    if isfield(variant, 'dwell_turn')
        state.dwell_turn = variant.dwell_turn;
        state.dwell_turn_steps = max(1, round(state.dwell_turn / state.Ts));
    end
    if isfield(variant, 'tau_theta')
        state.tau_theta = variant.tau_theta;
        state.alpha_theta = state.Ts / (state.tau_theta + state.Ts);
    end
end

function stats = compute_run_metrics(sim)
    % 针对单条 run 的预测与真值, 计算准确率、延时、抖动等指标
    time = sim.time;
    N = numel(time);
    start_idx = min(N, sim.seq_len + 1);
    idx = start_idx:N;
    if isempty(idx)
        idx = 1:N;
    end
    duration = time(end) - time(idx(1));
    stats.acc_main = mean(sim.pred_main(idx) == sim.truth_main(idx));
    stats.acc_turn = mean(sim.pred_turn(idx) == sim.truth_turn(idx));
    slope_mask = (sim.truth_main(idx) == 4);
    if any(slope_mask)
        stats.theta_mae = mean(abs(sim.pred_theta(idx(slope_mask)) - sim.truth_theta(idx(slope_mask))));
    else
        stats.theta_mae = NaN;
    end
    stats.delay_slope = compute_delay(sim.truth_main, sim.pred_main, time, 4);
    stats.delay_slip = compute_delay(sim.truth_main, sim.pred_main, time, 2);
    stats.delay_turn = compute_delay(sim.truth_turn, sim.pred_turn, time, 1); % left turn
    if isempty(idx) || numel(idx) < 2
        stats.flip_rate = 0;
    else
        flips = sum(sim.pred_main(idx(2:end)) ~= sim.pred_main(idx(1:end-1)));
        stats.flip_rate = flips / max(duration, eps);
    end
end

function delay = compute_delay(truth_signal, pred_signal, time, target_label)
    % 计算指定标签首次出现的时间差, 若缺失则返回 NaN
    truth_idx = find(truth_signal == target_label, 1, 'first');
    pred_idx = find(pred_signal == target_label, 1, 'first');
    if isempty(truth_idx) || isempty(pred_idx)
        delay = NaN;
    else

```

```

        delay = time(pred_idx) - time(truth_idx);
    end
end

function metrics = aggregate_metrics(run_stats, variant, cfg)
    % 将多条 run 的结果取平均, 并给出综合得分
    fields = {'acc_main', 'acc_turn', 'theta_mae', 'delay_slope', 'delay_slip', 'delay_turn', 'flip_rate'};
    aggregated = struct();
    for i = 1:numel(fields)
        f = fields{i};
        values = [run_stats.(f)];
        aggregated.(f) = mean(values, 'omitnan');
    end
    metrics = aggregated;
    metrics.variant = variant;
    metrics.theta_mae_deg = rad2deg(metrics.theta_mae);
    metrics.flip_rate_per_min = metrics.flip_rate * 60;
    metrics.delay_combined = mean([metrics.delay_slope, metrics.delay_slip], 'omitnan');
    if isnan(metrics.delay_combined)
        metrics.delay_combined = 0;
    end
    metrics.score = compute_score(metrics, cfg.weights);
end

function score = compute_score(m, w)
    % 根据设定权重计算综合得分, 越大越好
    score = w.acc_main * m.acc_main + ...
            w.acc_turn * m.acc_turn - ...
            w.delay * max(m.delay_combined, 0) - ...
            w.theta * max(m.theta_mae_deg, 0) - ...
            w.flips * max(m.flip_rate_per_min, 0);
end

function key = variant_key(v)
    % 构造缓存键, 确保不同组合唯一
    key = sprintf('dm%.3f_dt%.3f_tt%.3f', v.dwell_main, v.dwell_turn, v.tau_theta);
end

function scoreboard = collect_metrics(metrics_map)
    % 将 map 中的结构转为数组, 便于排序
    keys = metrics_map.keys;
    vals = cellfun(@(k) metrics_map(k), keys, 'UniformOutput', false);
    scoreboard = [vals{:}];
end

function [selected_runs, meta] = select_runs_for_latency(runs, per_mode)
    % 按行驶模式均匀抽样, 保证各类场景均有覆盖
    num_runs = numel(runs);
    mode_names = cell(num_runs, 1);
    for i = 1:num_runs
        mode_names{i} = get_scene_name(runs(i));
    end
    unique_modes = unique(mode_names);
    idx_list = [];
    for m = 1:numel(unique_modes)

```

```

        mask = strcmp(mode_names, unique_modes{m});
        mode_idx = find(mask);
        take = mode_idx(1:min(per_mode, numel(mode_idx)));
        idx_list = [idx_list; take(:)]; %#ok<AGROW>
    end
    idx_list = unique(idx_list);
    selected_runs = runs(idx_list);
    meta = struct('indices', idx_list, 'num_modes', numel(unique_modes));
end

function name = get_scene_name(source)
% 递归解析 run 元数据中的场景名称，兼容多种字段
default_name = 'run_unknown';
name = default_name;
text = extract_scene_text(source);
if ~isempty(text)
    name = text;
    return;
end
if ~isstruct(source)
    return;
end
fields = {'scene', 'path_type', 'scene_name', 'mode'};
for i = 1:numel(fields)
    if isfield(source, fields{i})
        text = extract_scene_text(source.(fields{i}));
        if ~isempty(text)
            name = text;
            return;
        end
    end
end
nested = {'meta', 'path_params'};
for i = 1:numel(nested)
    if isfield(source, nested{i})
        name = get_scene_name(source.(nested{i}));
        if ~strcmp(name, default_name)
            return;
        end
    end
end
end

function text = extract_scene_text(value)
% 提取字符串表示，兼容 char/string/cellstr
text = '';
if ischar(value)
    text = strtrim(value);
elseif isstring(value) && isscalar(value)
    text = strtrim(char(value));
elseif iscell(value) && numel(value)==1
    text = extract_scene_text(value{1});
end
end

```



```

===== FILE: src/tests/test_gru_filter_constants.m =====
% =====
% 文件名: test_gru_filter_constants.m
% 版本号: V1.0
% 最后修改时间: 2025-11-26
% 作者: LPV-MPC Project
% 功能描述:
%   在手动修改 GRU 输入侧一阶低通 (tau_accel_lp) 与差分滤波 (tau_diff)
%   时间常数后, 快速对在线推理性能做一次离线 run 级评估。
%   该脚本不会自动改写数据集/模型, 只会基于当前模型覆写 scaler
%   中的滤波时间常数, 然后调用 GRU_state_classifier 计算指标。
%
% 使用方法:
%   1. 根据需要在 "配置" 区域填写 override 数值 (单位: 秒)。
%   如果留空 ([ ]), 脚本会直接采用当前模型中的时间常数。
%   2. 运行本脚本: >> test_gru_filter_constants
%   3. 评估结果会打印在命令行, 并保存到 cfg.results_dir。
%
% 注意事项:
%   - 如需彻底生效, 应在修改完时间常数后重新执行
%     GRU_prepare_dataset → GRU_train.m, 以便 scaler / 模型内的数值同步。
%   - 本脚本仅用于实验性对比, 覆盖范围为 `cfg.runs_per_mode`
%     指定的每种行驶模式样本, 运行代价可控。
% =====

clear; clc;

root = project_root();
data_models_dir = fullfile(root, 'data', 'models');
data_gru_dir = fullfile(root, 'data', 'gru');
default_results_dir = results_dir('gru/filter_constant_eval');
fprintf('\n===== \n');
fprintf('GRU 滤波时间常数评估 (test_gru_filter_constants)\n');
fprintf('===== \n');

%% 配置
cfg = struct();
cfg.model_file = fullfile(data_models_dir, 'GRU_model.mat');
cfg.raw_data_file = fullfile(data_gru_dir, 'GRU_train_data_full.mat');
cfg.results_dir = default_results_dir;
cfg.runs_per_mode = 10; % 每个行驶模式抽取的 run 数量
cfg.tau_accel_lp_override = []; % 例如 0.30 (秒); 为空则使用模型值
cfg.tau_diff_override = []; % 例如 0.25 (秒); 为空则使用模型值
cfg.save_timeseries = false; % 若需调试可开启, 保存预测/真值轨迹

if ~exist(cfg.results_dir, 'dir')
    mkdir(cfg.results_dir);
end

%% 加载模型、数据并应用 override
fprintf('[1/4] 加载模型与数据...\n');
params = parameters();
model_data = load(cfg.model_file, 'model');
model = model_data.model;
raw = load(cfg.raw_data_file, 'data');

```

```

all_runs = raw.data.runs;

if ~isempty(cfg.tau_accel_lp_override)
    model.scaler.tau_accel_lp = cfg.tau_accel_lp_override;
    fprintf(' - 覆写 tau_accel_lp = %.3f s\n', model.scaler.tau_accel_lp);
elseif isfield(model.scaler, 'tau_accel_lp')
    fprintf(' - 使用模型中的 tau_accel_lp = %.3f s\n', model.scaler.tau_accel_lp);
else
    warning('model.scaler 缺少 tau_accel_lp (将 fallback 至 0.4s)');
end

if ~isempty(cfg.tau_diff_override)
    model.scaler.tau_diff = cfg.tau_diff_override;
    fprintf(' - 覆写 tau_diff = %.3f s\n', model.scaler.tau_diff);
elseif isfield(model.scaler, 'tau_diff')
    fprintf(' - 使用模型中的 tau_diff = %.3f s\n', model.scaler.tau_diff);
else
    warning('model.scaler 缺少 tau_diff (将 fallback 至 0.3s)');
end

[selected_runs, selection_meta] = select_runs_for_filter_eval(all_runs, cfg.runs_per_mode);
if isempty(selected_runs)
    error('未选到任何 run, 请检查原始数据。');
end
fprintf(' - 选取 %d 条 run, 覆盖 %d 种行驶模式\n', numel(selected_runs), selection_meta.num_modes);

%% 逐 run 仿真与统计
fprintf('\n[2/4] 逐 run 仿真...\n');
run_stats = arrayfun(@(r) simulate_run(r, params, model, cfg), selected_runs);

%% 聚合指标
fprintf('\n[3/4] 聚合指标...\n');
metrics = aggregate_metrics(run_stats);
print_metrics_table(metrics);

%% 保存结果
fprintf('\n[4/4] 保存结果...\n');
timestamp = datestr(now, 'yyyymmdd_HHMMSS');
summary = struct();
summary.cfg = cfg;
summary.metrics = metrics;
summary.selection_meta = selection_meta;
summary.run_stats = run_stats;
summary.model_scaler = model.scaler;
summary_file = fullfile(cfg.results_dir, sprintf('filter_eval_%s.mat', timestamp));
save(summary_file, '-struct', 'summary');
fprintf(' ✓ 已保存至 %s\n', summary_file);

fprintf('\n 评估完成。可结合不同的 tau_* override 数值多次运行本脚本, 比较输出指标。 \n');

%% =====
function [selected_runs, meta] = select_runs_for_filter_eval(runs, per_mode)
% 与 test_gru_latency 同理: 尽量覆盖不同场景
if per_mode <= 0
    selected_runs = runs;

```

```

        meta.num_modes = numel(unique({runs.scene}));
        return;
    end
    groups = struct();
    for i = 1:numel(runs)
        scene = get_scene_name(runs(i));
        if ~isfield(groups, scene)
            groups.(scene) = [];
        end
        groups.(scene) = [groups.(scene), i];
    end
    scenes = fieldnames(groups);
    selected_idx = [];
    for i = 1:numel(scenes)
        idx = groups.(scenes{i});
        take = min(numel(idx), per_mode);
        selected_idx = [selected_idx, idx(1:take)]; %#ok<AGROW>
    end
    selected_runs = runs(selected_idx);
    meta.num_modes = numel(scenes);
end

function name = get_scene_name(run)
    name = 'unknown';
    if isfield(run, 'scene') && ~isempty(run.scene)
        name = run.scene;
    elseif isfield(run, 'meta') && isfield(run.meta, 'scene')
        name = run.meta.scene;
    end
end

function stats = simulate_run(run, params, model, cfg)
% 复用 GRU_state_classifier 的 init/update, 返回关键指标
    state = GRU_state_classifier('init', params, model);
    N = size(run.y_raw, 1);
    out_main = zeros(N,1);
    out_turn = zeros(N,1);
    out_theta = zeros(N,1);
    for k = 1:N
        y = run.y_raw(k,:);
        [state, out] = GRU_state_classifier('update', state, y);
        out_main(k) = out.label_main;
        out_turn(k) = out.label_turn;
        out_theta(k) = out.theta_hat;
    end
    stats.scene = get_scene_name(run);
    stats.time = run.t;
    stats.truth_main = run.label_main;
    stats.truth_turn = run.label_turn;
    stats.truth_theta = run.theta;
    stats.pred_main = out_main;
    stats.pred_turn = out_turn;
    stats.pred_theta = out_theta;
    stats.seq_len = state.seq_len;
    stats = compute_metrics(stats);
end

```

```

if cfg.save_timeseries
    stats.timeseries = struct('time', stats.time, ...
        'pred_main', out_main, 'truth_main', run.label_main, ...
        'pred_turn', out_turn, 'truth_turn', run.label_turn, ...
        'pred_theta', out_theta, 'truth_theta', run.theta);
end
end

function stats = compute_metrics(sim)
    time = sim.time;
    N = numel(time);
    start_idx = min(N, sim.seq_len + 1);
    idx = start_idx:N;
    if isempty(idx)
        idx = 1:N;
    end
    duration = time(idx(end)) - time(idx(1));
    stats = struct();
    stats.scene = sim.scene;
    stats.acc_main = mean(sim.pred_main(idx) == sim.truth_main(idx));
    stats.acc_turn = mean(sim.pred_turn(idx) == sim.truth_turn(idx));
    slope_mask = sim.truth_main(idx) == 4;
    if any(slope_mask)
        stats.theta_mae = mean(abs(sim.pred_theta(idx(slope_mask)) - sim.truth_theta(idx(slope_mask))));
    else
        stats.theta_mae = NaN;
    end
    stats.delay_slope = compute_delay(sim.truth_main, sim.pred_main, time, 4);
    stats.delay_slip = compute_delay(sim.truth_main, sim.pred_main, time, 2);
    stats.delay_turn = compute_delay(sim.truth_turn, sim.pred_turn, time, 1);
    if numel(idx) < 2
        stats.flip_rate = 0;
    else
        flips = sum(sim.pred_main(idx(2:end)) ~= sim.pred_main(idx(1:end-1)));
        stats.flip_rate = flips / max(duration, eps);
    end
end

function delay = compute_delay(truth_signal, pred_signal, time, label)
    truth_idx = find(truth_signal == label, 1, 'first');
    pred_idx = find(pred_signal == label, 1, 'first');
    if isempty(truth_idx) || isempty(pred_idx)
        delay = NaN;
    else
        delay = time(pred_idx) - time(truth_idx);
    end
end

function metrics = aggregate_metrics(run_stats)
    fields = {'acc_main', 'acc_turn', 'theta_mae', 'delay_slope', 'delay_slip', 'delay_turn', 'flip_rate'};
    metrics = struct();
    for i = 1:numel(fields)
        f = fields{i};
        values = [run_stats.(f)];
        metrics.(f) = mean(values, 'omitnan');
    end
end

```

```

end
metrics.theta_mae_deg = rad2deg(metrics.theta_mae);
metrics.flip_rate_per_min = metrics.flip_rate * 60;
end

function print_metrics_table(m)
    fprintf('\n=== 滤波参数评估指标 ===\n');
    fprintf('主分类准确率      : %.3f\n', m.acc_main);
    fprintf('转弯准确率      : %.3f\n', m.acc_turn);
    fprintf('θ MAE (deg)      : %.3f\n', m.theta_mae_deg);
    fprintf('Slope 延迟 (s) : %.3f\n', m.delay_slope);
    fprintf('Slip 延迟 (s) : %.3f\n', m.delay_slip);
    fprintf('Turn 延迟 (s) : %.3f\n', m.delay_turn);
    fprintf('抖动 (次/分钟) : %.2f\n', m.flip_rate_per_min);
end

===== FILE: src/tests/test_agv_open_loop.m =====
% =====
% 文件名: test_agv_open_loop.m
% 功能描述: 开环验证 AGV 物理模型在不同参数下的响应 (优化版)
% 修改说明:
%   1. 加入空气阻力到理论计算
%   2. 转向测试中施加维持速度的前馈力, 避免速度衰减
%   3. 提高加速度计算的数值稳定性
% =====

clc; clear;
fprintf('=== AGV 模型开环物理验证脚本 (V2.0 优化版) ===\n');

%% 加载参数
params = parameters();
Ts = params.Ts;
m = params.mass;
g = params.gravity;
c_r = params.rolling_resistance;
rho_air = params.air_density;
CdA = params.drag_coefficient_area;

% 计算有效质量 (包含电机/车轮转动惯量)
r_wheel = params.wheel_radius;
n_gear = params.gear_ratio;
Jw = params.wheel_inertia;
Jm = params.motor_inertia;
m_eff = m + 2*(Jw + Jm*n_gear^2) / r_wheel^2;

fprintf('系统参数配置:\n');
fprintf(' - 采样步长 Ts: %.3f s\n', Ts);
fprintf(' - 车辆质量 m: %.1f kg\n', m);
fprintf(' - 有效质量 m_eff: %.2f kg\n', m_eff);

%% 测试函数定义
run_sim = @(t_end, u_fixed, theta_g, x0) simulate_core(t_end, u_fixed, theta_g, x0, params);

%% [A] 静态平衡测试
fprintf('\n[A] 静态平衡测试 (零输入)...');

```

```

t_test = 5.0;
[t, states] = run_sim(t_test, [0; 0], 0, zeros(8,1));
v_peak = max(abs(states(:, 4)));
if v_peak < 1e-4
    fprintf(' [PASS] 峰值速度漂移: %.2e m/s\n', v_peak);
else
    fprintf(' [FAIL] 峰值速度漂移: %.2e m/s\n', v_peak);
end

%% [B] 恒力加速测试
fprintf(' [B] 恒力加速测试 (F_cmd=200N)...');
F_input = 200;
[t, states] = run_sim(2.0, [F_input; 0], 0, zeros(8,1));

% 计算理论平均加速度 (考虑滚阻和平均速度下的空气阻力)
v_avg = mean(states(:,4));
F_res_avg = m * g * c_r + 0.5 * rho_air * CdA * v_avg^2;
a_theory = (F_input - F_res_avg) / m_eff;

% 使用 1.0s - 2.0s 之间的平均加速度
idx = t >= 1.0;
v_range = states(idx, 4);
t_range = t(idx);
% 线性回归求斜率
p = polyfit(t_range, v_range, 1);
accel_actual = p(1);
err_pct = abs(accel_actual - a_theory) / a_theory * 100;

if err_pct < 10
    fprintf(' [PASS] 理论加速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
else
    fprintf(' [FAIL] 理论加速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
end

%% [C] 转向响应测试 (C1-C4)
% 在测试中施加平衡力, 保持速度接近 v0=1m/s
v_test = 1.0;
F_bal = m*g*c_r + 0.5*rho_air*CdA*v_test^2;

test_cases = {
    'C1', '左转-低速', 0.1, 10.0;
    'C2', '左转-S弯', 0.37, 2.7; % 原 0.5→0.37 rad/s (匹配 S 弯)
    'C3', '右转-低速', -0.1, 10.0;
    'C4', '右转-S弯', -0.37, 2.7; % 原 -0.5→-0.37 rad/s (匹配 S 弯)
    'C5', '右转-第一弯', -0.39, 2.56 % 匹配 industrial 路径 50-54s 的第一个右转 (63°/4s*1.43≈0.39)
};

for i = 1:size(test_cases, 1)
    id = test_cases{i,1};
    desc = test_cases{i,2};
    w_cmd = test_cases{i,3};
    R_theory = test_cases{i,4};

    fprintf('[%s] %s (w_cmd=%.2f rad/s)... \n', id, desc, w_cmd);

```

```

x0 = zeros(8,1); x0(4) = v_test;
[t, states] = run_sim(8.0, [F_bal; w_cmd], 0, x0);

% 取最后 2 秒的均值
final_idx = t > 6.0;
w_actual = mean(states(final_idx, 5));
v_actual = mean(states(final_idx, 4));
delta_lf_actual = mean(states(final_idx, 6));
delta_rr_actual = mean(states(final_idx, 7));
beta_actual = mean(states(final_idx, 8));

R_actual = abs(v_actual / (w_actual + eps));
err_pct = abs(R_actual - R_theory) / R_theory * 100;
w_tracking_ratio = w_actual / w_cmd * 100; % 角速度跟踪比 (%)

% 诊断输出
fprintf('      诊断: v=%.3f m/s, w=%.4f rad/s (跟踪率=%.1f%%)\n', v_actual, w_actual, w_tracking_ratio);
fprintf('           $\delta_{lf}$ =%.2f°,  $\delta_{rr}$ =%.2f°,  $\beta$ =%.3f°\n', rad2deg(delta_lf_actual),
rad2deg(delta_rr_actual), rad2deg(beta_actual));
fprintf('          理论 R=%.1fm, 实际 R=%.2fm, 误差=%.1f%%\n', R_theory, R_actual, err_pct);

if err_pct < 15 && sign(w_actual) == sign(w_cmd)
    fprintf('      [PASS]\n');
else
    fprintf('      [FAIL] <- 可能原因: ');
    if abs(w_tracking_ratio) < 90
        fprintf('角速度跟踪不足\n');
    elseif abs(v_actual - v_test) > 0.1
        fprintf('速度偏离目标\n');
    else
        fprintf('需进一步分析\n');
    end
end
end

%% [D] 坡度阻力测试
fprintf('[D] 坡度阻力测试 (theta=5 deg)...');
theta_deg = 5;
theta_rad = deg2rad(theta_deg);
x0 = zeros(8,1); x0(4) = 1.5;
[t, states] = run_sim(1.0, [0; 0], theta_rad, x0);

v_avg = mean(states(:,4));
a_theory = (-m*g*sin(theta_rad) - m*g*cos(theta_rad)*c_r - 0.5*rho_air*CdA*v_avg^2) / m_eff;

p = polyfit(t, states(:,4), 1);
accel_actual = p(1);
err_pct = abs(accel_actual - a_theory) / abs(a_theory) * 100;

if err_pct < 10
    fprintf(' [PASS] 理论减速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
else
    fprintf(' [FAIL] 理论减速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
end

%% [E] 稳态速度测试

```

```

fprintf('[E] 稳态速度测试 (平衡阻力)...');
v_target = 1.2; % 提高到 1.2m/s
F_maintain = m*g*c_r + 0.5*rho_air*CdA*v_target^2;

x0 = zeros(8,1); x0(4) = v_target;
[t, states] = run_sim(10.0, [F_maintain; 0], 0, x0);
% 使用基于时间的索引, 取最后 2 秒
steady_idx = t > (t(end) - 2.0);
v_steady = mean(states(steady_idx, 4));
v_rmse = sqrt(mean((states(steady_idx,4) - v_target).^2));

if v_rmse < 0.05
    fprintf(' [PASS] 目标=%.2f, 稳态=%.2f, RMSE=%.3f m/s\n', v_target, v_steady, v_rmse);
else
    fprintf(' [FAIL] 目标=%.2f, 稳态=%.2f, RMSE=%.3f m/s\n', v_target, v_steady, v_rmse);
end

fprintf('\n 所有开环验证完成。 \n');

% [F] Mamba 新增输出变量验证 (V4.3)
fprintf('\n=== [F] Mamba 新增输出变量验证 (V4.3) ===\n');
fprintf(' 清除 persistent 变量...\n');
clear output_eq_ref; % 清除 persistent 变量

% 场景 F1: 平地直行 - slip_ratio 应接近 0, gyro_y 应接近 0, accel_y 应接近 0
fprintf('[F1] 平地直行 - 验证新输出变量...\n');
x0_f1 = zeros(8,1); x0_f1(4) = 1.0; % 初始速度 1 m/s
[t_f1, states_f1, outputs_f1] = simulate_core_with_outputs(5.0, [50; 0], 0, x0_f1, params);

slip_lf_mean = mean(outputs_f1(end-100:end, 32));
slip_rr_mean = mean(outputs_f1(end-100:end, 33));
gyro_y_mean = mean(outputs_f1(end-100:end, 10));
accel_y_mean = mean(outputs_f1(end-100:end, 34));

fprintf('    slip_ratio_lf: %.4f (期望≈0)\n', slip_lf_mean);
fprintf('    slip_ratio_rr: %.4f (期望≈0)\n', slip_rr_mean);
fprintf('    gyro_y_meas:   %.4f rad/s (期望≈0)\n', gyro_y_mean);
fprintf('    accel_y_meas:  %.4f m/s² (期望≈0)\n', accel_y_mean);

if abs(slip_lf_mean) < 0.1 && abs(slip_rr_mean) < 0.1 && abs(gyro_y_mean) < 0.05 && abs(accel_y_mean) < 0.2
    fprintf('    [PASS]\n');
else
    fprintf('    [FAIL] 某些输出超出预期范围\n');
end

% 场景 F2: 平地转弯 - accel_y 应有明显变化
fprintf('[F2] 平地转弯 - 验证横向加速度...\n');
clear output_eq_ref;
x0_f2 = zeros(8,1); x0_f2(4) = 1.0;
[t_f2, states_f2, outputs_f2] = simulate_core_with_outputs(5.0, [50; 0.2], 0, x0_f2, params);

accel_y_turn_mean = mean(abs(outputs_f2(end-100:end, 34)));
gyro_y_turn_mean = mean(abs(outputs_f2(end-100:end, 10)));

fprintf('    accel_y_meas 均值: %.4f m/s² (期望 > 0)\n', accel_y_turn_mean);

```



```

fprintf('    gyro_y_meas 均值: %.4f rad/s (期望=0, 平地无坡度变化)\n', gyro_y_turn_mean);

if accel_y_turn_mean > 0.01
    fprintf('    [PASS] 转弯时横向加速度有响应\n');
else
    fprintf('    [FAIL] 转弯时横向加速度无明显响应\n');
end

% 场景 F3: 坡度切换 - gyro_y 应在切换时有脉冲
fprintf('[F3] 坡度切换 - 验证俯仰角速度...\n');
clear output_eq_ref;
x0_f3 = zeros(8,1); x0_f3(4) = 1.0;

% 模拟坡度切换: 前 2 秒平地, 后 3 秒上坡
t_switch = 2.0;
theta_slope = deg2rad(5);
[t_f3, ~, outputs_f3] = simulate_core_with_slope_switch(5.0, [100; 0], x0_f3, params, t_switch,
theta_slope);

% 找到切换时刻附近的 gyro_y 峰值
switch_idx = find(t_f3 >= t_switch, 1, 'first');
gyro_y_around_switch = outputs_f3(max(1,switch_idx-5):min(end,switch_idx+10), 10);
gyro_y_peak = max(abs(gyro_y_around_switch));
gyro_y_before = mean(abs(outputs_f3(1:switch_idx-10, 10)));
gyro_y_after = mean(abs(outputs_f3(switch_idx+50:end, 10)));

fprintf('    坡度切换前 gyro_y 均值: %.4f rad/s\n', gyro_y_before);
fprintf('    坡度切换时 gyro_y 峰值: %.4f rad/s\n', gyro_y_peak);
fprintf('    坡度切换后 gyro_y 均值: %.4f rad/s\n', gyro_y_after);

if gyro_y_peak > gyro_y_before * 5 && gyro_y_peak > 0.01
    fprintf('    [PASS] 坡度切换时 gyro_y 出现脉冲\n');
else
    fprintf('    [FAIL] 坡度切换时 gyro_y 无明显脉冲\n');
end

fprintf('\n=== Mamba 新增输出变量验证完成 ===\n');

%% [G] 极限爬坡测试 (10 度坡)
fprintf('\n[G] 极限爬坡测试 (10 度坡, F_cmd=600N)...\n');
theta_climb = deg2rad(10);
F_climb = 600;
x0_climb = zeros(8,1); % Still zero speed start
[t_g, states_g] = run_sim(5.0, [F_climb; 0], theta_climb, x0_climb);

v_final_g = mean(states_g(end-50:end, 4));
dist_g = states_g(end, 1); % X position

fprintf('    最终速度: %.3f m/s\n', v_final_g);
fprintf('    爬升距离: %.3f m\n', dist_g);

% 判定: 速度 > 0.1 m/s (能爬上去)
if v_final_g > 0.1
    fprintf('    [PASS] 车辆成功爬上 10 度坡\n');
else
    fprintf('    [FAIL] 车辆爬坡失败 (速度不足)\n');
end

```

end

%% 核心仿真循环函数

```
function [t_vec, x_history] = simulate_core(t_end, u_fixed, theta_g, x0, params)
    Ts = params.Ts;
    N = round(t_end / Ts);
    t_vec = (0:Ts:N*Ts)';
    x_history = zeros(N+1, 8);
    x_history(1, :) = x0';

    xk = x0;
    for k = 1:N
        xk = state_eq_ref(xk, u_fixed, theta_g, params);
        x_history(k+1, :) = xk';
    end
end
```

%% 核心仿真循环函数（含输出采集）

```
function [t_vec, x_history, y_history] = simulate_core_with_outputs(t_end, u_fixed, theta_g, x0, params)
    Ts = params.Ts;
    N = round(t_end / Ts);
    t_vec = (0:Ts:N*Ts)';
    x_history = zeros(N+1, 8);
    y_history = zeros(N+1, 34); % V4.3: 34 维输出
    x_history(1, :) = x0';
    y_history(1, :) = output_eq_ref(x0, u_fixed, theta_g, params)';

    xk = x0;
    for k = 1:N
        xk = state_eq_ref(xk, u_fixed, theta_g, params);
        yk = output_eq_ref(xk, u_fixed, theta_g, params);
        x_history(k+1, :) = xk';
        y_history(k+1, :) = yk';
    end
end
```

%% 坡度切换仿真函数

```
function [t_vec, x_history, y_history] = simulate_core_with_slope_switch(t_end, u_fixed, x0, params,
t_switch, theta_after)
    Ts = params.Ts;
    N = round(t_end / Ts);
    t_vec = (0:Ts:N*Ts)';
    x_history = zeros(N+1, 8);
    y_history = zeros(N+1, 34);
    x_history(1, :) = x0';

    xk = x0;
    for k = 1:N
        t_current = (k-1) * Ts;
        if t_current < t_switch
            theta_g = 0;
        else
            theta_g = theta_after;
        end

        xk = state_eq_ref(xk, u_fixed, theta_g, params);
    end
end
```

```

        yk = output_eq_ref(xk, u_fixed, theta_g, params);
        x_history(k+1, :) = xk';
        y_history(k+1, :) = yk';
    end
end

===== FILE: src/tests/test_industrial_open_loop_items.m =====
% =====
% 文件名: test_industrial_open_loop_items.m
% 功能描述: 按 Industrial 参考路径(ref.meta.segments)分项进行开环工况验证 (含串联/动态评估)
% 说明:
%   - 目标是验证“模型本体”在典型工况下能否产生合理响应 (不依赖 MPC 纠偏)
%   - 支持两种模式:
%       * independent: 每段独立 (稳态评估)
%       * serial: 串联段落 (动态与耦合评估)
%   - 每段使用 ref 信号统计得到的 v/omega/theta 进行开环指令
%   - 参考数据来源: data/paths/path_industrial.mat
% =====

clc; clear;
fprintf('=== Industrial Path 分项开环验证脚本 (V1.1) ===\n');

%% 加载参数
params = parameters();
Ts = params.Ts;
m = params.mass;
g = params.gravity;
c_r = params.rolling_resistance;
rho_air = params.air_density;
CdA = params.drag_coefficient_area;

% 有效质量 (包含电机/车轮转动惯量)
r_wheel = params.wheel_radius;
n_gear = params.gear_ratio;
Jw = params.wheel_inertia;
Jm = params.motor_inertia;
m_eff = m + 2*(Jw + Jm*n_gear^2) / r_wheel^2;

%% 读取当前 Industrial 路径元数据 (基于脚本位置定位工程根目录)
script_dir = fileparts(mfilename('fullpath'));
project_root = fileparts(fileparts(script_dir));
p = fullfile(project_root, 'data', 'paths', 'path_industrial.mat');
S = load(p);
assert(isfield(S, 'ref'), 'path_industrial.mat 中未找到 ref');
ref = S.ref;
assert(isfield(ref, 'meta') && isfield(ref.meta, 'segments'), 'ref.meta.segments 不存在');
segments = ref.meta.segments;

assert(isfield(ref, 't') && isfield(ref, 'v_ref') && isfield(ref, 'omega_ref') && isfield(ref, 'theta_ref'), ...
    'ref 中缺少 t/v_ref/omega_ref/theta_ref');
ref_t = ref.t(:);
ref_v = ref.v_ref(:);
ref_omega = ref.omega_ref(:);
ref_theta = ref.theta_ref(:);

```

```

assert(isfield(ref, 'X_ref') && isfield(ref, 'Y_ref') && isfield(ref, 'psi_ref'), ...
    'ref 中缺少 X_ref/Y_ref/psi_ref');
ref_X = ref.X_ref(:);
ref_Y = ref.Y_ref(:);
ref_psi = ref.psi_ref(:);

% 参考加速度（用于前馈力）
ref_vdot = gradient(ref_v, ref_t);

fprintf('ref.meta.version: %s\n', safeGetField(ref.meta, 'version', 'N/A'));
fprintf('ref.meta.generation_time: %s\n', safeGetField(ref.meta, 'generation_time', 'N/A'));

%% 测试模式与覆盖策略（可用环境变量覆盖）
% 可选：
% INDUSTRIAL_OPEN_LOOP_MODE = serial / independent
% INDUSTRIAL_OPEN_LOOP_INCLUDE_ALL = 1/0
% INDUSTRIAL_OPEN_LOOP_INCLUDE_ISOLATION = 1/0
mode = getConfigStr('INDUSTRIAL_OPEN_LOOP_MODE', 'serial');
include_all = getConfigBool('INDUSTRIAL_OPEN_LOOP_INCLUDE_ALL', true);
include_isolation = getConfigBool('INDUSTRIAL_OPEN_LOOP_INCLUDE_ISOLATION', true);

% 指令生成方式：
% ref_replay: 段内按 ref(t) 逐步回放（F/omega/theta 均可时变）
% piecewise_constant: 段内用常值近似（更接近“工况稳态验证”）
command_profile = getConfigStr('INDUSTRIAL_OPEN_LOOP_COMMAND_PROFILE', 'ref_replay');

%% 从 segments 构造待测段列表
cases = {};
for i = 1:numel(segments)
    seg = segments(i);
    typ = char(seg.type);
    desc = char(seg.desc);

    dur = seg.t_end - seg.t_start;
    if dur <= 0
        continue;
    end

    if ~include_all
        isKey = false;
        switch typ
            case {'left_turn', 'right_turn'}
                isKey = true;
            case 'straight'
                if abs(seg.slope) > deg2rad(0.1)
                    isKey = true;
                end
        end
        if ~isKey
            continue;
        end
    end

    if ~include_isolation && contains(desc, '隔离')
        continue;
    end
end

```

```

end

cases(end+1, :) = {i, typ, desc, dur, seg.t_start, seg.t_end}; %#ok<SAGROW>
end

fprintf('\n 待测段数量: %d (mode=%s, include_all=%d) \n', size(cases,1), mode, include_all);

%% 测试配置
min_run_time = 6.0;      % independent 模式每段最短仿真时长
steady_window = 2.0;     % 末尾稳态统计窗口
min_rmse_eval_dur = 0.5; % 段太短时不做 RMSE 判定 (用端点/符号替代)

% 容差 (可根据模型实际调整)
v_tol = 0.10;            % m/s (稳态均值)
omega_rel_tol = 0.15;    % 相对误差 (稳态均值)
omega_abs_tol = 0.02;    % rad/s (omega_cmd=0 时)
v_rmse_tol = 0.12;       % m/s (动态 RMSE)
omega_rmse_tol = 0.08;   % rad/s (动态 RMSE)
v_end_tol = 0.20;        % m/s (短段/加减速段端点速度误差)
omega_end_tol = 0.06;    % rad/s (短段端点角速度误差)

% 诊断阈值 (用于全程统计)
slip_ratio_warn = 0.20;  % 纵向滑移率警戒阈值
beta_warn_deg = 5.0;     % 质心侧偏角警戒阈值 [deg]

%% 逐段执行
results =
struct('idx',{},'type',{},'desc',{},'pass',{},'v_mean',{},'omega_mean',{},'theta_g',{},'note',{},'v_rmse',{
},'omega_rmse',{},'v_end_err',{},'omega_end_err',{});

% 串联模式初值
x0_serial = zeros(8,1);
x0_serial(4) = 0; % 从静止开始

% 全程记录
t_global = [];
x_global = [];
u_global = [];
theta_global = [];

for k = 1:size(cases,1)
    idx = cases{k,1};
    typ = cases{k,2};
    desc = cases{k,3};
    dur = cases{k,4};
    t_start = cases{k,5};
    t_end_ref = cases{k,6};

    [v_ref_mean, omega_ref_mean, theta_ref_mean, vdot_ref_mean] = refSegmentStats(ref_t, ref_v, ref_omega,
ref_theta, t_start, t_end_ref);
    [v_ref_steady_mean, omega_ref_steady_mean] = refSteadyStats(ref_t, ref_v, ref_omega, t_start, t_end_ref,
steady_window);

    if strcmpi(mode, 'independent')
        t_end = max(dur, min_run_time);
        x0 = zeros(8,1);
        x0(4) = max(v_ref_mean, 0);
    else
        t_end = max(dur, Ts);
    end
end

```

```

    x0 = x0_serial;
end

if strcmpi(command_profile, 'ref_replay')
    % 段内按 ref(t) 回放: F/omega/theta 都随时间变化
    u_fun = @(t_local) [ ...
        computeForceFromRef(t_start + t_local, ref_t, ref_v, ref_vdot, ref_theta, m, g, c_r, rho_air,
CdA, m_eff); ...
        interp1(ref_t, ref_omega, t_start + t_local, 'linear', 'extrap') ...
    ];
    theta_fun = @(t_local) interp1(ref_t, ref_theta, t_start + t_local, 'linear', 'extrap');
    [t, states] = simulate_core_tv(t_end, u_fun, theta_fun, x0, params);
    theta_g = theta_ref_mean;
    omega_cmd = omega_ref_mean;
else
    % 段内常值近似: 使用段均值 (更接近“稳态工况”)
    theta_g = theta_ref_mean;
    omega_cmd = omega_ref_mean;
    F_ff = computeForceToTrackSpeedAndAccel(v_ref_mean, vdot_ref_mean, theta_g, m, g, c_r, rho_air, CdA,
m_eff);
    u_fun = @(t_local) [F_ff; omega_cmd];
    theta_fun = @(t_local) theta_g;
    [t, states] = simulate_core(t_end, [F_ff; omega_cmd], theta_g, x0, params);
end

if strcmpi(mode, 'serial')
    x0_serial = states(end, :);
end

% 记录本段输入/坡度 (用于全程统计)
u_hist = zeros(numel(t), 2);
theta_hist = zeros(numel(t), 1);
for ii = 1:numel(t)
    u_hist(ii, :) = u_fun(t(ii));
    theta_hist(ii) = theta_fun(t(ii));
end

t_abs = t_start + t;
if isempty(t_global)
    keep_idx = 1:numel(t_abs);
else
    keep_idx = 2:numel(t_abs); % 避免段首重复点
end
t_global = [t_global; t_abs(keep_idx)]; %#ok<AGROW>
x_global = [x_global; states(keep_idx, :)]; %#ok<AGROW>
u_global = [u_global; u_hist(keep_idx, :)]; %#ok<AGROW>
theta_global = [theta_global; theta_hist(keep_idx)]; %#ok<AGROW>

% 动态对齐 (与 ref 段内曲线比较)
t_eval = min(dur, t(end));
ref_t_local = ref_t - t_start;
idx_ref = ref_t >= t_start & ref_t <= t_end_ref;
t_ref_seg = ref_t_local(idx_ref);
v_ref_seg = ref_v(idx_ref);
omega_ref_seg = ref_omega(idx_ref);

t_sim = t(t <= t_eval);

```

```

v_sim = states(1:numel(t_sim), 4);
omega_sim = states(1:numel(t_sim), 5);

v_ref_interp = interp1(t_ref_seg, v_ref_seg, t_sim, 'linear', 'extrap');
omega_ref_interp = interp1(t_ref_seg, omega_ref_seg, t_sim, 'linear', 'extrap');

v_rmse = sqrt(mean((v_sim - v_ref_interp).^2));
omega_rmse = sqrt(mean((omega_sim - omega_ref_interp).^2));

v_end_err = abs(v_sim(end) - v_ref_interp(end));
omega_end_err = abs(omega_sim(end) - omega_ref_interp(end));

% 末尾稳态统计（用于与 ref 同窗口对比）
steady_idx = t > (t(end) - min(steady_window, t(end)));
v_mean = mean(states(steady_idx, 4));
omega_mean = mean(states(steady_idx, 5));

% 判据（按段类型/段长做差异化）
is_short = dur < min_rmse_eval_dur;
is_accel = strcmpi(typ, 'accel');
is_decel = strcmpi(typ, 'decel');

pass_dyn = (v_rmse <= v_rmse_tol) && (omega_rmse <= omega_rmse_tol);
pass_end = (v_end_err <= v_end_tol) && (omega_end_err <= omega_end_tol);

% 稳态均值判据：只用于非加减速且段长足够的情况
if ~(is_accel || is_decel) && ~is_short
    % 注意：必须与 ref 的“段末同一时间窗口”对齐，否则会把段内过渡当作稳态误差
    pass_v = abs(v_mean - v_ref_steady_mean) <= v_tol;
    omega_err = omega_mean - omega_ref_steady_mean;
    omega_allow = max(omega_abs_tol, omega_rel_tol * abs(omega_ref_steady_mean));

    if abs(omega_ref_steady_mean) < 0.05
        % 近零角速度不宜用相对误差（会被放大）
        pass_w = abs(omega_err) <= omega_abs_tol;
    else
        pass_w = abs(omega_err) <= omega_allow && sign(omega_mean) == sign(omega_ref_steady_mean);
    end
else
    pass_v = true;
    pass_w = true;
end

if is_short
    pass = pass_end;
else
    pass = pass_v && pass_w && pass_dyn;
end

note = '';
if ~pass
    if is_short
        note = [note, sprintf('短段端点误差(v=%.3f, omega=%.4f); ', v_end_err, omega_end_err)];
    else
        if ~pass_v

```

```

        note = [note, sprintf('v 偏差(%.3f vs %.3f); ', v_mean, v_ref_steady_mean)];
    end
    if ~pass_w
        note = [note, sprintf('omega 偏差(%.4f vs %.4f); ', omega_mean, omega_ref_steady_mean)];
    end
    if ~pass_dyn
        note = [note, sprintf('RMSE(v=%.3f, omega=%.4f); ', v_rmse, omega_rmse)];
    end
end
end

results(end+1) = struct( ...
    'idx', idx, ...
    'type', typ, ...
    'desc', desc, ...
    'pass', pass, ...
    'v_mean', v_mean, ...
    'omega_mean', omega_mean, ...
    'theta_g', theta_g, ...
    'note', note, ...
    'v_rmse', v_rmse, ...
    'omega_rmse', omega_rmse, ...
    'v_end_err', v_end_err, ...
    'omega_end_err', omega_end_err);

fprintf(['%02d] %-10s | theta=%+5.2fdeg | omega_cmd=%+7.4f | v=%.3f | omega=%.4f | RMSE(v=%.3f, w=%.4f)
| %s\n', ...
    idx, typ, rad2deg(theta_g), omega_cmd, v_mean, omega_mean, v_rmse, omega_rmse,
    ternary(pass, 'PASS', 'FAIL')));
if ~pass
    fprintf('    note: %s\n', note);
    fprintf('    desc: %s\n', desc);
end
end

%% 汇总
pass_count = sum([results.pass]);
fprintf('\n=== 汇总: %d/%d 通过 ===\n', pass_count, numel(results));

fail_idx = find(~[results.pass]);
if ~isempty(fail_idx)
    fprintf('未通过段索引: ');
    fprintf('%d ', [results(fail_idx).idx]);
    fprintf('\n');
    fprintf('建议: 优先查看这些段对应的 turn/slope 参数、以及模型的 omega 跟踪机制/输入限幅。 \n');
else
    fprintf('所有关键工况段均满足开环验收判据。 \n');
end

%% 全程统计: 轨迹漂移 / 侧偏 / 滑移 / 网格覆盖
if ~isempty(t_global)
    % 轨迹误差 (对齐参考)
    X_ref_interp = interp1(ref_t, ref_X, t_global, 'linear', 'extrap');
    Y_ref_interp = interp1(ref_t, ref_Y, t_global, 'linear', 'extrap');
    psi_ref_interp = interp1(ref_t, ref_psi, t_global, 'linear', 'extrap');

    X_err = x_global(:,1) - X_ref_interp;

```



```

Y_err = x_global(:,2) - Y_ref_interp;
psi_err = wrapToPi(x_global(:,3) - psi_ref_interp);

fprintf('\n=== 全程轨迹误差统计 ===\n');
fprintf('RMSE: X=%.3f m, Y=%.3f m, psi=%.3f deg\n', rms(X_err), rms(Y_err), rad2deg(rms(psi_err)));
fprintf('终点误差: X=%.3f m, Y=%.3f m, psi=%.3f deg\n', X_err(end), Y_err(end), rad2deg(psi_err(end)));

% 侧偏角与滑移率统计
beta = x_global(:,8);
beta_warn = abs(beta) > deg2rad(beta_warn_deg);

slip_lf = nan(size(t_global));
slip_rr = nan(size(t_global));
for ii = 1:numel(t_global)
    y = output_eq_ref(x_global(ii,:), u_global(ii,:), theta_global(ii), params);
    slip_lf(ii) = y(32);
    slip_rr(ii) = y(33);
end
slip_warn = (abs(slip_lf) > slip_ratio_warn) | (abs(slip_rr) > slip_ratio_warn);

fprintf('\n=== 侧偏/滑移统计 ===\n');
fprintf('beta 超限(>%.1fdeg): %d / %d (%.2f%%)\n', beta_warn_deg, sum(beta_warn), numel(beta_warn),
100*mean(beta_warn));
fprintf('slip_ratio 超限(>%.2f): %d / %d (%.2f%%)\n', slip_ratio_warn, sum(slip_warn), numel(slip_warn),
100*mean(slip_warn));

% 输入饱和统计（按电机电流限制推算）
F_wheel_max = params.current_limit * params.motor_torque_constant * (params.gear_efficiency *
params.gear_ratio) / max(params.wheel_radius, 1e-6);
F_cmd_max_total = 2 * F_wheel_max;
F_cmd = u_global(:,1);
F_sat = abs(F_cmd) > F_cmd_max_total;
fprintf('\n=== 输入饱和统计 ===\n');
fprintf('F_cmd 超限: %d / %d (%.2f%%), F_cmd_max=%.1f N\n', sum(F_sat), numel(F_sat), 100*mean(F_sat),
F_cmd_max_total);

% LPV 网格覆盖
grid_file = fullfile(project_root, 'models', 'lin_agv_db.mat');
if exist(grid_file, 'file')
    G = load(grid_file);
    [V_grid, W_grid, T_grid] = extractLPVGrid(G);
    if ~isempty(V_grid) && ~isempty(W_grid) && ~isempty(T_grid)
        v_in = x_global(:,4);
        w_in = x_global(:,5);
        t_in = theta_global;
        out_v = v_in < min(V_grid) | v_in > max(V_grid);
        out_w = w_in < min(W_grid) | w_in > max(W_grid);
        out_t = t_in < min(T_grid) | t_in > max(T_grid);
        fprintf('\n=== LPV 网格覆盖统计 ===\n');
        fprintf('V 超网格: %d / %d (%.2f%%)\n', sum(out_v), numel(out_v), 100*mean(out_v));
        fprintf('W 超网格: %d / %d (%.2f%%)\n', sum(out_w), numel(out_w), 100*mean(out_w));
        fprintf('T 超网格: %d / %d (%.2f%%)\n', sum(out_t), numel(out_t), 100*mean(out_t));
    end
end

end

%% ===== Local Functions =====

function [v_mean, omega_mean, theta_mean, vdot_mean] = refSegmentStats(t, v_ref, omega_ref, theta_ref,
t_start, t_end)

```

```

idx = t >= t_start & t <= t_end;
if ~any(idx)
    v_mean = 0; omega_mean = 0; theta_mean = 0; vdot_mean = 0;
    return;
end

v_seg = v_ref(idx);
omega_seg = omega_ref(idx);
theta_seg = theta_ref(idx);
t_seg = t(idx);

v_mean = mean(v_seg);
omega_mean = mean(omega_seg);
theta_mean = mean(theta_seg);

if numel(v_seg) >= 2
    vdot = diff(v_seg) ./ diff(t_seg);
    vdot_mean = mean(vdot);
else
    vdot_mean = 0;
end
end

function [v_steady_mean, omega_steady_mean] = refSteadyStats(t, v_ref, omega_ref, t_start, t_end,
steady_window)
% 段末稳态窗口必须限制在该段内部，避免把上一段尾巴混进来
t0 = max(t_start, t_end - steady_window);
idx = t >= t0 & t <= t_end;
if ~any(idx)
    v_steady_mean = v_ref(find(t <= t_end, 1, 'last'));
    omega_steady_mean = omega_ref(find(t <= t_end, 1, 'last'));
    return;
end
v_steady_mean = mean(v_ref(idx));
omega_steady_mean = mean(omega_ref(idx));
end

function F = computeForceToTrackSpeedAndAccel(v, vdot, theta_g, m, g, c_r, rho_air, CdA, m_eff)
% 目标：用前馈力近似跟踪速度与加速度
% F = m_eff*vdot + F_rr + F_grade + F_aero
F = m_eff * vdot + m*g*cos(theta_g)*c_r + m*g*sin(theta_g) + 0.5*rho_air*CdA*v^2;
end

function F = computeForceFromRef(t_abs, ref_t, ref_v, ref_vdot, ref_theta, m, g, c_r, rho_air, CdA, m_eff)
v = interp1(ref_t, ref_v, t_abs, 'linear', 'extrap');
vdot = interp1(ref_t, ref_vdot, t_abs, 'linear', 'extrap');
theta_g = interp1(ref_t, ref_theta, t_abs, 'linear', 'extrap');
F = m_eff * vdot + m*g*cos(theta_g)*c_r + m*g*sin(theta_g) + 0.5*rho_air*CdA*v^2;
end

function out = ternary(cond, a, b)
if cond
    out = a;
else
    out = b;
end
end

```

```

end

function v = getConfigStr(envName, defaultValue)
    v = getenv(envName);
    if isempty(v)
        v = defaultValue;
    end
end

function v = getConfigBool(envName, defaultValue)
    s = getenv(envName);
    if isempty(s)
        v = defaultValue;
        return;
    end
    v = any(strcmpi(s, {'1','true','yes','on'}));
end

function v = safeGetField(s, fieldName, defaultValue)
    if isstruct(s) && isfield(s, fieldName)
        v = s.(fieldName);
    else
        v = defaultValue;
    end
end

function [t_vec, x_history] = simulate_core(t_end, u_fixed, theta_g, x0, params)
    Ts = params.Ts;
    N = round(t_end / Ts);
    t_vec = (0:Ts:N*Ts)';
    x_history = zeros(N+1, 8);
    x_history(1, :) = x0';

    xk = x0;
    for k = 1:N
        xk = state_eq_ref(xk, u_fixed, theta_g, params);
        x_history(k+1, :) = xk';
    end
end

function [t_vec, x_history] = simulate_core_tv(t_end, u_fun, theta_fun, x0, params)
    Ts = params.Ts;
    N = round(t_end / Ts);
    t_vec = (0:Ts:N*Ts)';
    x_history = zeros(N+1, 8);
    x_history(1, :) = x0';

    xk = x0;
    for k = 1:N
        t_local = (k-1) * Ts;
        u = u_fun(t_local);
        theta_g = theta_fun(t_local);
        xk = state_eq_ref(xk, u, theta_g, params);
        x_history(k+1, :) = xk';
    end
end

```

```

end

function [V_grid, W_grid, T_grid] = extractLPVGrid(G)
    V_grid = [];
    W_grid = [];
    T_grid = [];

    if isfield(G, 'V_grid'), V_grid = G.V_grid; end
    if isfield(G, 'W_grid'), W_grid = G.W_grid; end
    if isfield(G, 'T_grid'), T_grid = G.T_grid; end

    if isempty(V_grid) || isempty(W_grid) || isempty(T_grid)
        % 尝试从结构体字段中递归提取
        names = fieldnames(G);
        for i = 1:numel(names)
            v = G.(names{i});
            if isstruct(v)
                if isempty(V_grid) && isfield(v, 'V_grid'), V_grid = v.V_grid; end
                if isempty(W_grid) && isfield(v, 'W_grid'), W_grid = v.W_grid; end
                if isempty(T_grid) && isfield(v, 'T_grid'), T_grid = v.T_grid; end
            end
        end
    end
end
end
end

```